# ALBALQA'A ABLIED UNIVERSITY

# FACULTY OF ENGINEERING TECHNOLOGY

# MECHATRONICS DEPARTMENT

# PROGRAMMABLE LOGIC CONTROLLER LAB

## Table of Contents

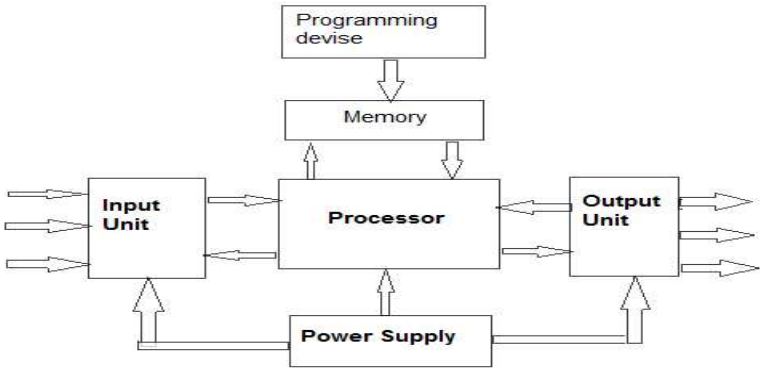
Introduction of PLC
Experiment 1
**Familiarization with the Micro-Win and S7- 200Simulation**
Experiment 2
**STEP 7 – MICRO/WIN and S7-200 Simulation Problems**
Experiment 3
**Timers**
Experiment 4
**Timer Problems**
Experiment 5
**Communication PLC Siemens S7-200 to PC and Micro-Win**
Experiment 6
**Counter**
Experiment 7
**Counter Problems**
Experiment 8
**PLC Wiring Input and Output**
Experiment 9
**LOGO LOGIC**
Experiment 10
**Examples**
Experiment 11
**Microbot MCC**
Experiment 12
**Mobile Robot**

# Introduction to PLC

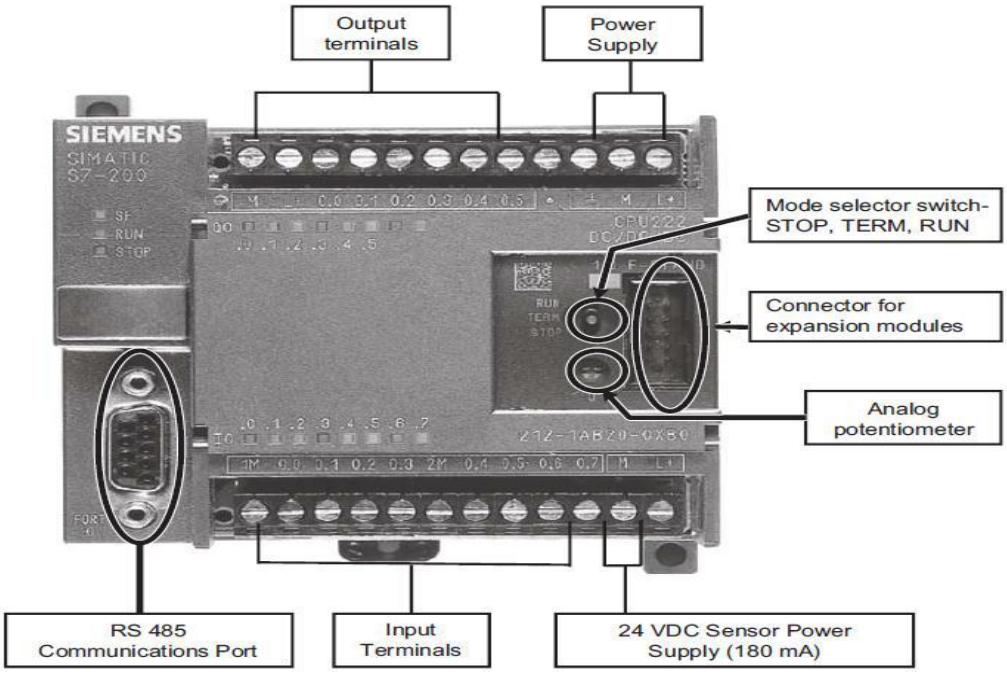
Controllers may consist of logical components and connections among them. Depending on the current logical value of input, output is produced to change the status of the system. PLC may realize such controllers.
Today, the command and feedback control systems of industrial automation systems are realized by programmable logic controllers (PLCs). Siemens Simatic S7-200 is one of the PLC brands widely used in industry.
In order for PLCs to work as controllers, they must be able to realize some functions. These functions are basic and combinational logic operations such as AND, OR, AND-NOT, OR-NOT, timer and counter operations. In addition to these, PLCs may have the ability to realize several transfer, mathematical, and PID operations.

## PLC consists:

**1- CPU** is the brain of PLC. It reads the input values from inputs, runs the program existed in the program memory and writes the output values to the output register.
**2- Power Supply** is needed to convert the mains a.c voltage to the low d.c voltage (5 V) necessary for the processor and the circuits in the input and output interface modules.
**3- Memory** is used to store different types of information in the binary structure form. The memory range of S7-200 is composed of three main parts as program, parameter, and retentive data fields.
**4- I/O units** provide communication between PLC control systems.
**5- Programming devise** is used to enter the required program into the memory of the processor.



The PLC System




# S7-200 PLC

## PLC Programming Language

- Ladder diagram(LAD)
- Statement List (STL)
- Functional Block(FB)

### The Advantages of PLC

1. Easy programming and Modification
2. Ease of fault detection and maintenance
3. Small size
4. Consume less electrical capacity
5. Speed of Response
6. PLC can be connected with many devices
7. Access to the reports when a fault tolerance by the connected computer with PLC

**How to work the PLC:**

The work of PLC depends on the Process of scan time of Program (scan time is the time needed to implementation of the program)

1. Check the status of the inputs if ON or OFF, Then store the case of inputs in Memory.
2. The CPU bring the case of inputs from memory and then implementation of the program stored
3. The CPU give new orders to the outputs.

**Experiment 1** **STEP 7 – MICRO/WIN and S7-200 Simulation**

### 1.1 OBJECTIVES

After successfully completing this laboratory, you should be able to:

- To convert a simple electrical ladder diagram to a PLC program.
- Know the difference between physical components and program components.
- To used Micro-Win program and S7-200 Simulation.
- Editing Ladder diagram.
- Testing Ladder diagram.
- Monitoring status by S7-200 Simulator.

### 1.2 Theory

Step7 – Micro/WIN makes programming of S7-200 easier. Programming of S7-200 by using Step 7 – Micro/WIN will be introduced in a simple example. In STEP 7-Micro/WIN, click on New option in the Project menu for new project. After composing program, click the Compile button on the taskbar. By clicking this button, software translates the program code block into machine language for execution by the CPU. A program cannot be downloaded to the CPU until it is compiled. If there is an error in your program, your program will not be compiled and software will warn you about the errors in your program. After correcting errors, try to compile again.

Inputs will be defined as [I0.0; I0.1; I0.2; I0.3; …..]

Outputs will be defined as [Q0.0 ;Q0.1;Q0.2; Q0.3; …….]

Flags will be defined as [M0.0; M0.1; M0.2; M0.3; …….]

I ----------→ input
Q ----------→ output
0 ----------→ byte
0, 1 , 2 ---→ bit

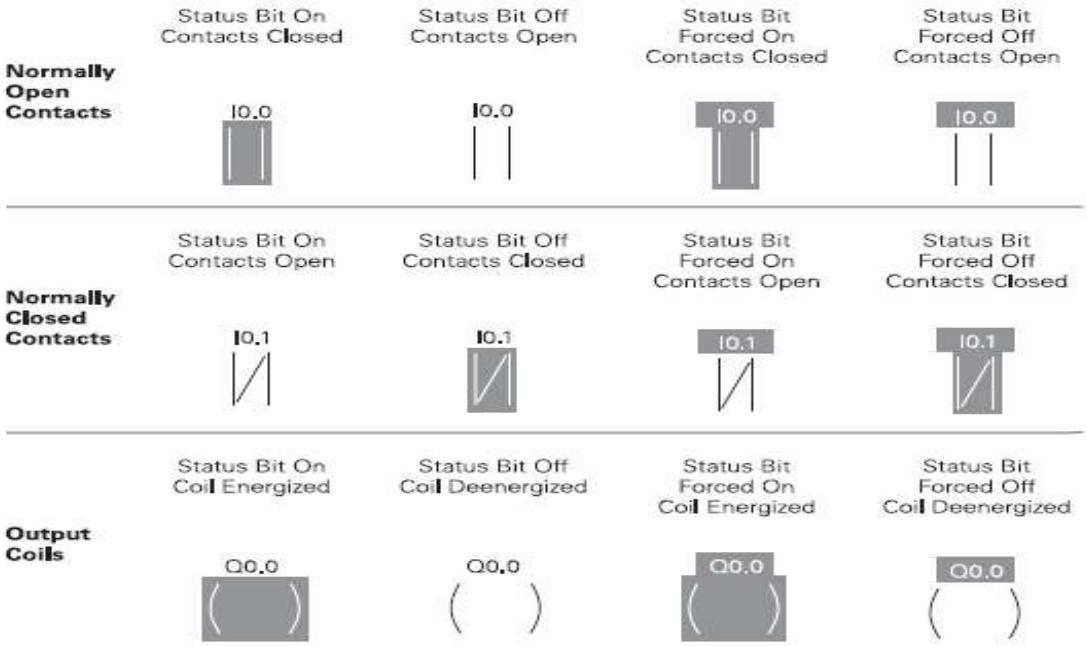
### 1.2 Equipments

1. Micro-Win Program
2. S7-200 Simulation

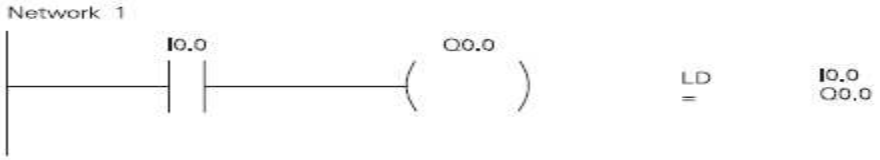
### 1.4 Program Instruction

A network of LAD is a row of connected elements that form a complete circuit between the left and right power rail. The left power rail represents the energized conductor whereas the right power rail represents the return path conductor of the circuit. Power flows from the left rail, through the closed contacts to the coils or boxes connected to the right power rail. You can then use the power flow to activate the outputs according to your program.

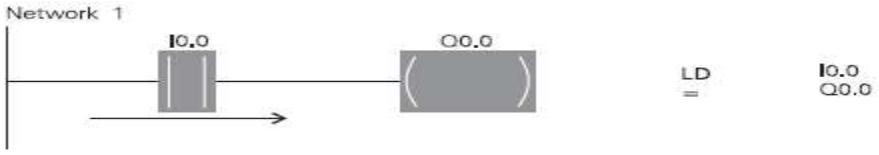
The following table shows the appearance of ladder elements in the Off, forced, and On condition.



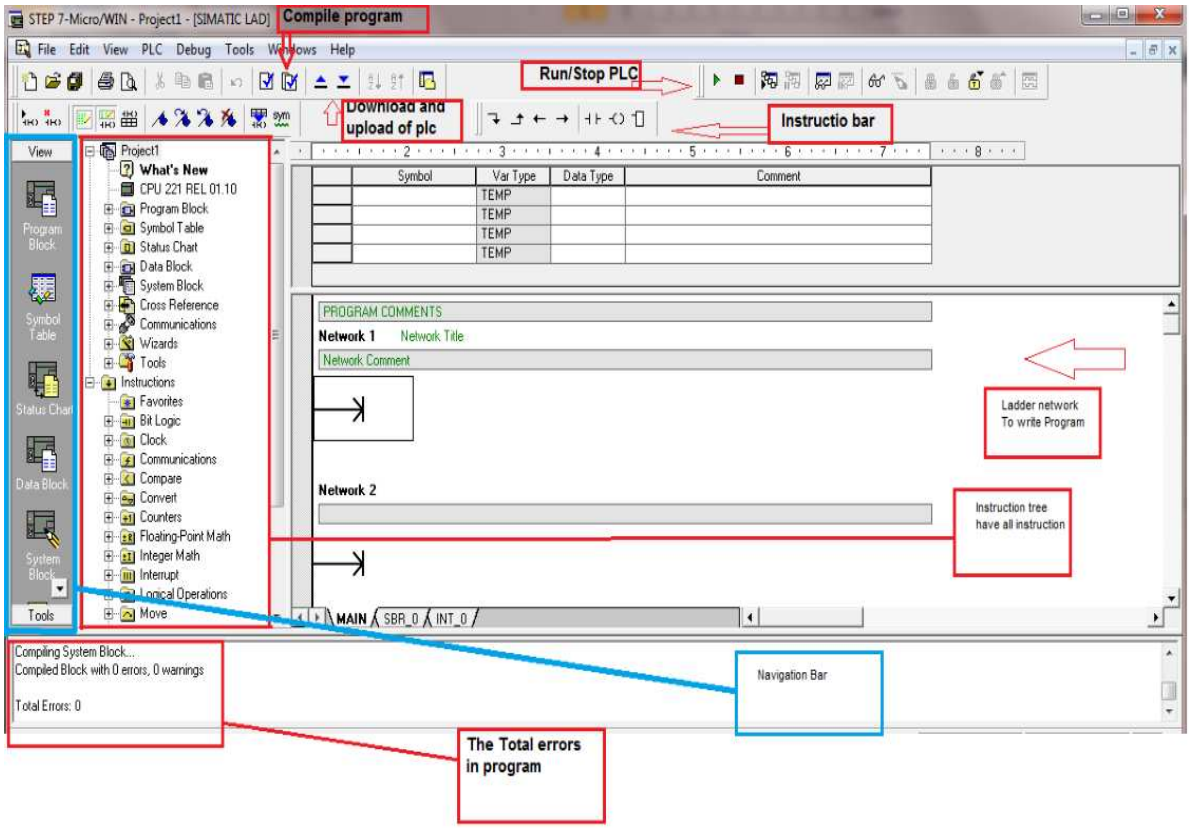

When the switch is open the CPU receives a logic 0 from input I0.0. The CPU sends a logic 0 to output Q0.0 and the light is off.



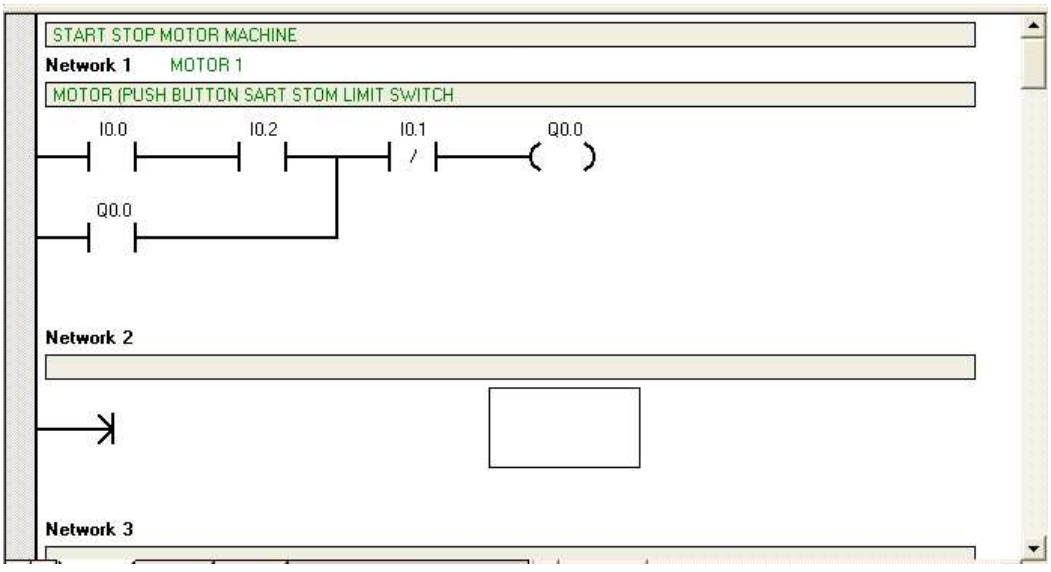

When the switch is closed the CPU receives a logic 1 from input I0.0. The CPU sends a logic 1 to output Q0.0, thus activating Q0.0. The light turns on.



## 1.5 Micro-Win Program




Step 7 Micro/WIN Start Window



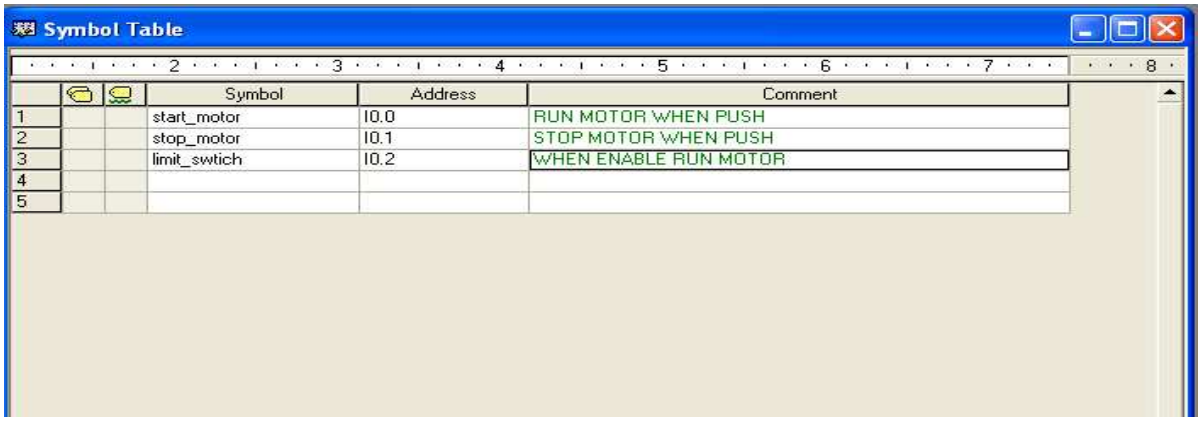

Example of ladder diagram

**The steps thinking in reviews of the PLC program**

1. The understanding and analysis of the industrial process to be the work of the program
2. The identification of the Input and Output in the table with the description
3. Division the industrial process to the parts and work program of each part

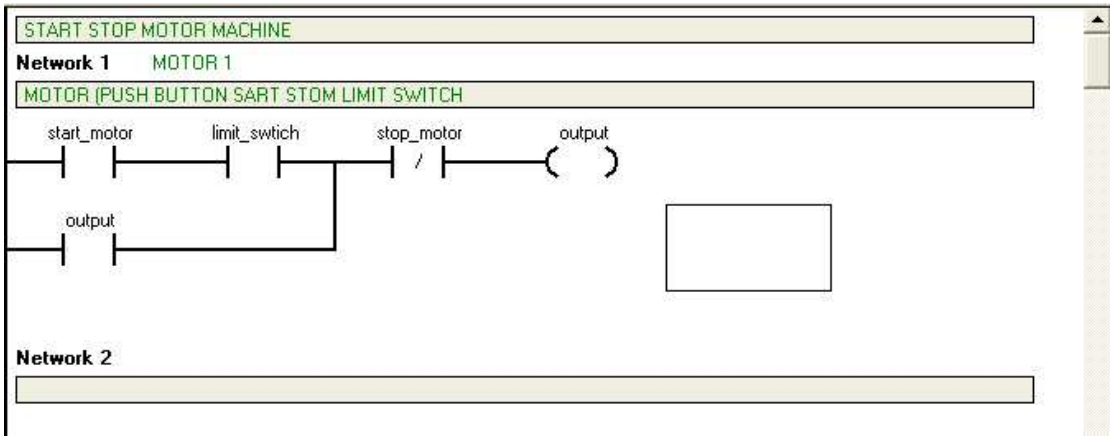
Write the symbol tables ( it easy to search process within the program)

View → component→ symbol table




Now, if we want appearance of detail the program of data entered by the symbol table is work the following:

View → symbol table → apply symbol to project




If we are to reverse previous situation

view→ symbolic addressing

After we write the program we want to compile the program for errors

From PLC → compile all

We note in the Output window, [total error : 0]

To save program

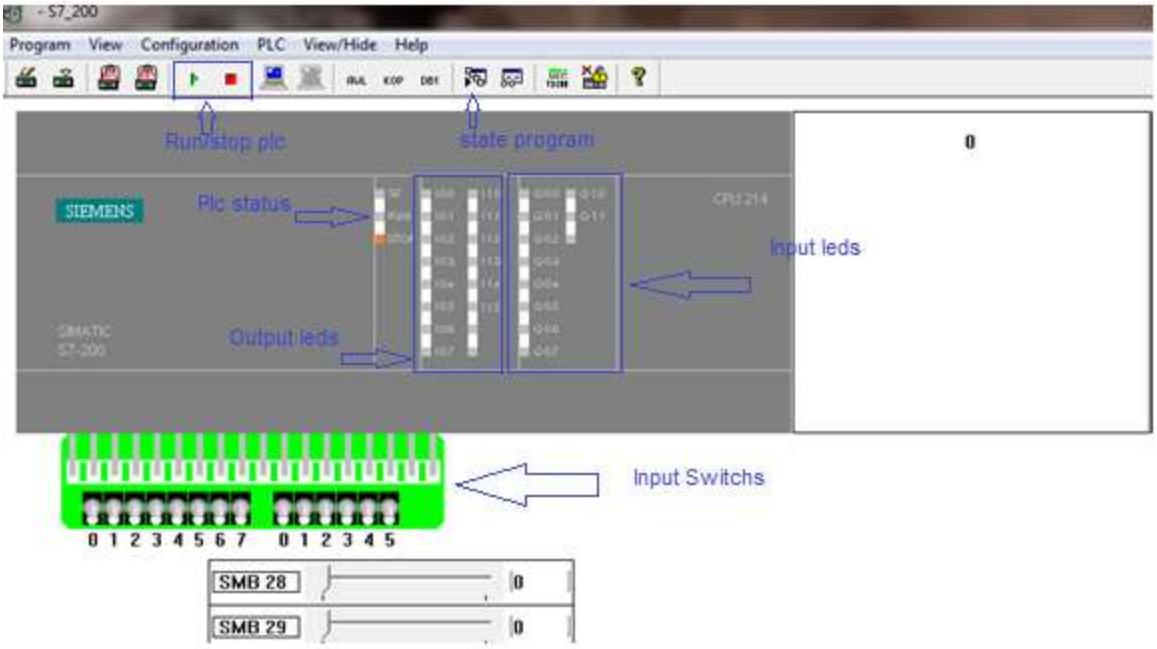
File→ Export [To save as (awl) file]

## 1.6 S7-200 Simulation

To  load the program to S7-200 simulator

Program → Load Program

Tow windows are open one the program in ladder diagram and the second in Instruction list.




S7-200 Simulation

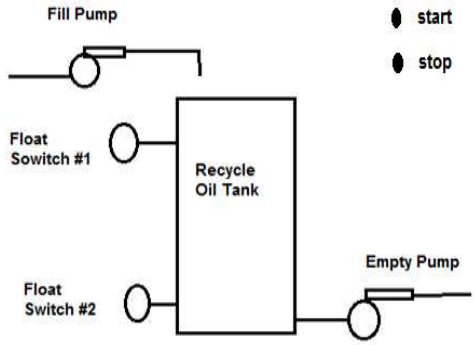
To Run / stop simulation of PLC  from PLC we press Run or stop.

**Experiment 2** **STEP 7 – MICRO/WIN and S7-200 Simulation Problems**

## Problem 1

In a recycle oil tank problem shown in the figure below, when the "Fill" button is pressed, the Fill Pump should run until Float Switch #1 is activated or the stop button is pressed. When the "Empty " button is pressed and Float Switch #2 is activated, the Empty Pump should run until Float Switch #2 is de-activated or the Stop button is pressed.

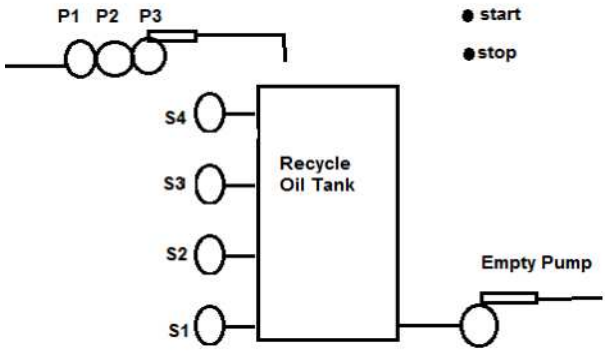
Write the ladder diagram and input/output table
Write the program in Instruction list.



## Problem 2

In the figure below, when we pressed start button the Fill Pump P1,P2,P3 are run.
When S2 is activated or the stop button pressed P1 Stop,
When S3 is activated or the stop button pressed P2 Stop ,
When S4 is activated or the stop button pressed P2 Stop ,
When S4 is activated the Empty Pump should run until S1 de-activated or stop button pressed.

Write the ladder diagram and input/output table
Write the program in Instruction list.



**Experiment 3** **Timers**

# Timers

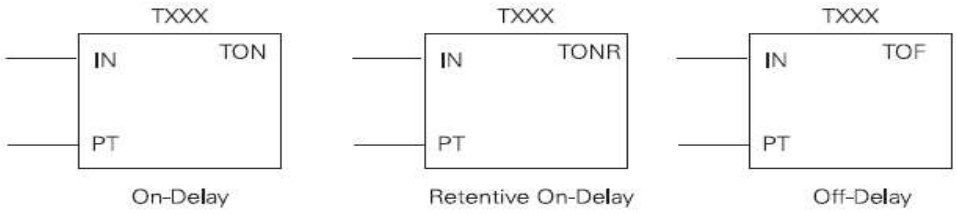
## 3.1 Objective

- To study the operation of different types of timers.
- To use the S7-200 timers in a process control.

## 3.2 Timer Instructions

### 3.2.1 On-Delay Timer / Retentive On-Delay Timer

The On-Delay Timer (TON) and Retentive On-Delay Timer (TONR) instructions count time when the enabling input is on. The timer number (Txx) determines the resolution of the timer, and the resolution is now shown in the instruction box.

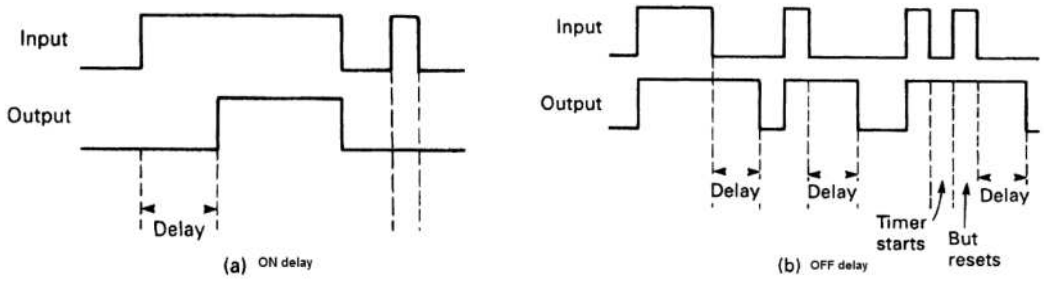



### 3.2.2 Off-Delay Timer

The Off-Delay Timer (TOF) is used to delay turning an output off for a fixed period of time after the input turns off.

The timer number (Txx) determines the resolution of the timer, and the resolution is now shown in the instruction box.

- You can use a TON for timing a single interval.
- You can use a TONR for accumulating a number of timed intervals.
- You can use a TOF for extending time past an off (or false) condition, such as for cooling a motor after it is turned off.



Operations of the Timer Instructions

| Type | Current >= Preset | State of the Enabling Input (IN) | Power Cycle/First Scan |
|---|---|---|---|
| TON | Timer bit on<br>Current continues counting to 32,767 | ON: Current value counts time<br>OFF: Timer bit off, current value = 0 | Timer bit off<br>Current value = 0 |
| TONR | Timer bit on<br>Current continues counting to 32,767 | ON: Current value counts time<br>OFF: Timer bit and current value maintain last state | Timer bit off<br>Current value can be maintained[1] |
| TOF | Timer bit off<br>Current = Preset, stops counting | ON: Timer bit on, current value = 0<br>OFF: Timer counts after on-to-off transition | Timer bit off<br>Current value = 0 |

The TON and TONR instructions count time when the enabling input is on. When the current value is equal to or greater than the preset time, the timer bit is on.

- The current value of a TON timer is cleared when the enabling input is off, whereas the current value of the TONR timer is maintained when the input is off.
- You can use the TONR timer to accumulate time when the input turns on and off. Use the Reset instruction (R) to clear the current value of the TONR.
- Both the TON and the TONR timers continue counting after the preset is reached, and they stop counting at the maximum value of 32,767 s.
- The TOF instruction is used to delay turning an output off for a fixed period of time after the input turns off. When the enabling input turns on, the timer bit turns on immediately, and the current value is set to 0. When the input turns off, the timer counts until the elapsed time reaches the preset time.
- When the preset is reached, the timer bit turns off and the current value stops incrementing; however, if the input turns on again before the TOF reaches the preset value, the timer bit remains on.
- The enabling input must make an on-to-off transition for the TOF to begin counting time intervals.

You can reset a TONR only by using the Reset (R) instruction. You can also use the Reset instruction to reset any TON or TOF. The Reset instruction performs

### 3.3 Determining the Resolution of the Timer

Timers count time intervals. The resolution (or time base) of the timer determines the amount of time in each interval. For example, a TON with a resolution of 10 ms counts the number of 10-ms intervals that elapse after the TON is enabled: a count of 50 on a 10-ms timer represents 500 ms. The SIMATIC timers are available in three resolutions: 1 ms, 10 ms, and 100 ms, the timer number determines the resolution of the timer.

Timer Numbers and Resolutions

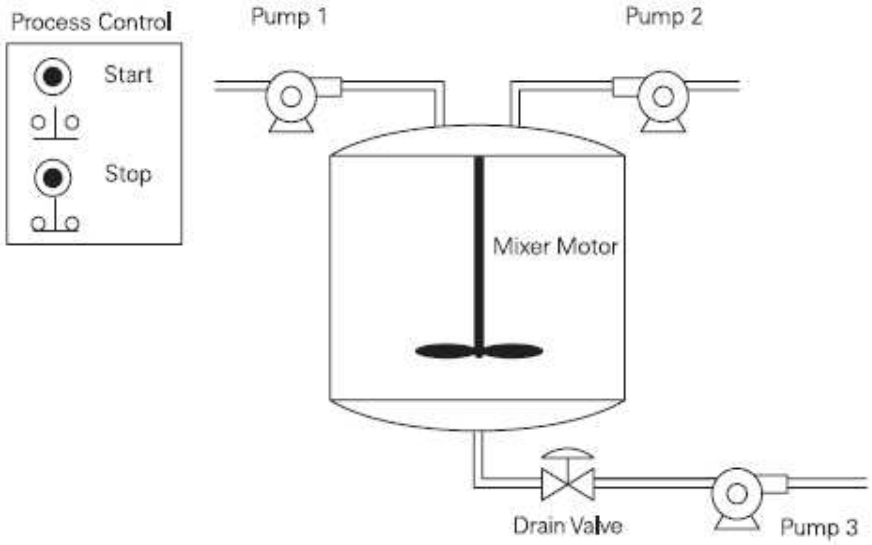
| Timer Type | Resolution | Maximum Value | Timer Number |
|---|---|---|---|
| TON<br>TOFF | 1ms | 32.767 seconds | T32,T64 |
| | 10ms | 327.67 seconds | T33-T36 , T97 – T100 |
| | 100ms | 3276.7 seconds | T33 – T63 , T101 – T255 |
| TONR | 1ms | 32.767 seconds | T0 , T64 |
| | 10ms | 327.67 seconds | T1-T4 , T65 – T68 |
| | 100ms | 3276.7 seconds | T5-T31 , T69-T95 |

**Problem 1**

A tank will be filled with two chemicals, mixed, and then drained. When the Start Button is pressed, the program starts pump 1 runs for 5 seconds, filling the tank with the first chemical, then shuts off. The program then starts pump 2, runs for 3 seconds filling the tank with the second chemical. After 3 seconds pump 2 shuts off. The program starts the mixer motor, and mixes the two chemicals for 60 seconds. The program then opens the drain valve, and starts pump 3.

Pump 3 shuts off after 8 seconds and the process stops. A manual Stop switch is also provided to stop process at any time .

- Write a ladder diagram for S7-200 PLC to control the system
- Input / output table
- Write program in instruction list

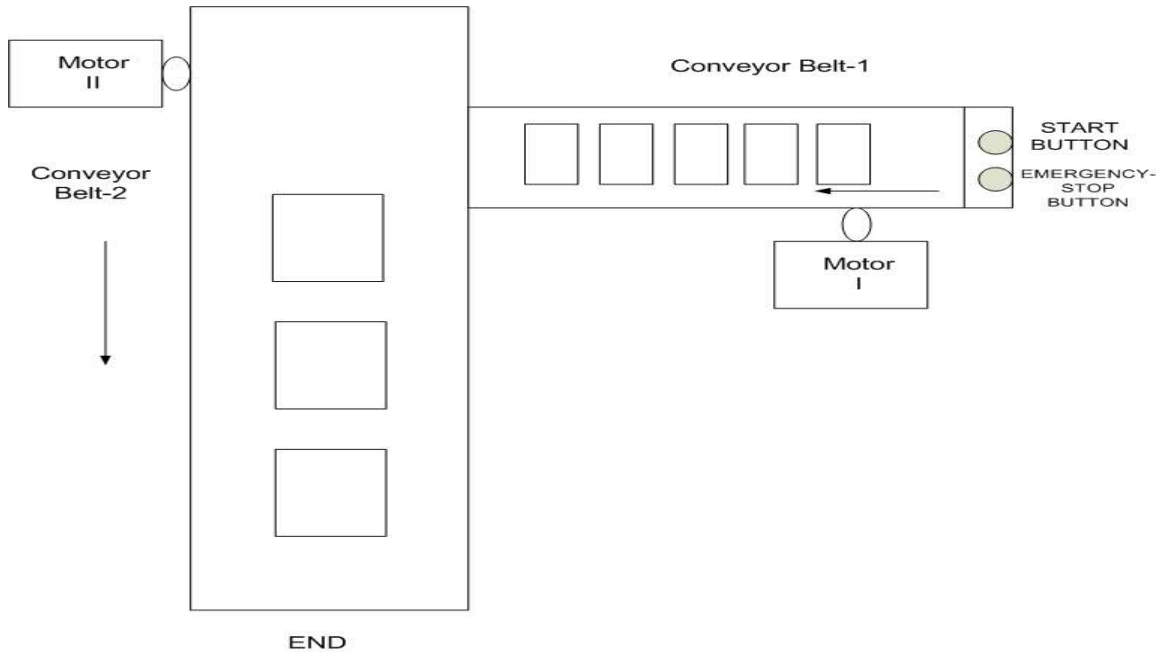


**Experiment 4** **Timers Problems**

**Problem 1**

The system to be controlled by PLC consists of two belts. If the Start button is pressed, Conveyor Belt-1 will begin to run. After 5 seconds Conveyor Belt-2 will be active. After the whole system runs for 15 seconds, Conveyor Belt-1 will stop. Then Conveyor Belt-2 continues to move for 5 seconds and then it will stop, too. Also the system can be reset by the emergency-stop button at any time.

- Write a ladder diagram for S7-200 PLC to control the system
- Input / output table
- Write program in instruction list



**Problem 2**

Control a traffic light system using plc with the following sequence:

1. The red light is on for a 5 seconds
2. The red light turns off and the yellow light is turned on for a 2 seconds
3. The yellow light turns off and the green light is turned on for a 4 seconds
4. The red light turns on again

**Experiment 5 Communication PLC Siemens S7-200 to PC and Micro-Win**

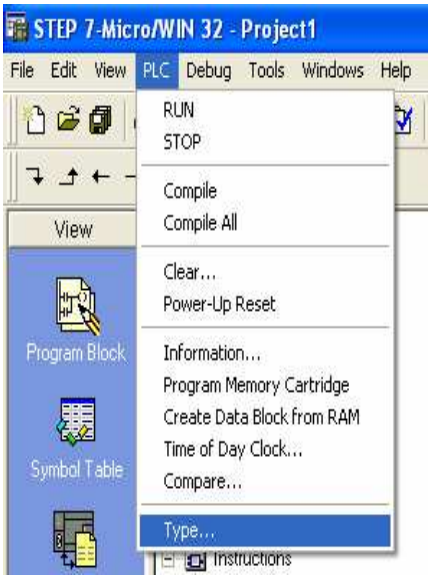
## 5.1 Communication PLC to PC

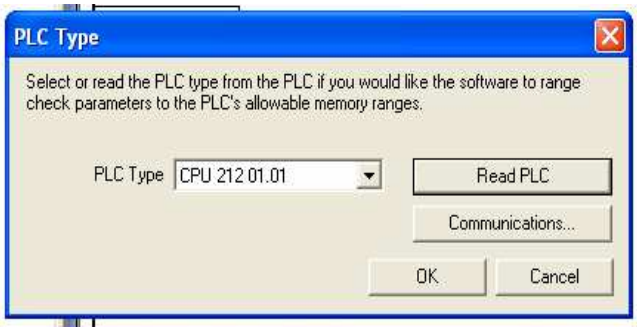
1. Siemens PLC used in this program: S7-200 CPU212

   a) Connect part number PC/PPI cable from PC to Siemens S7-200.

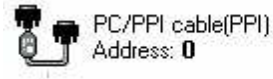
2. Launch Siemens programming software.

3. Select PLC – Type.
4. Next, click on the Read PLC button to read your PLC type.
5. Click on the Communications button and the settings should appear.

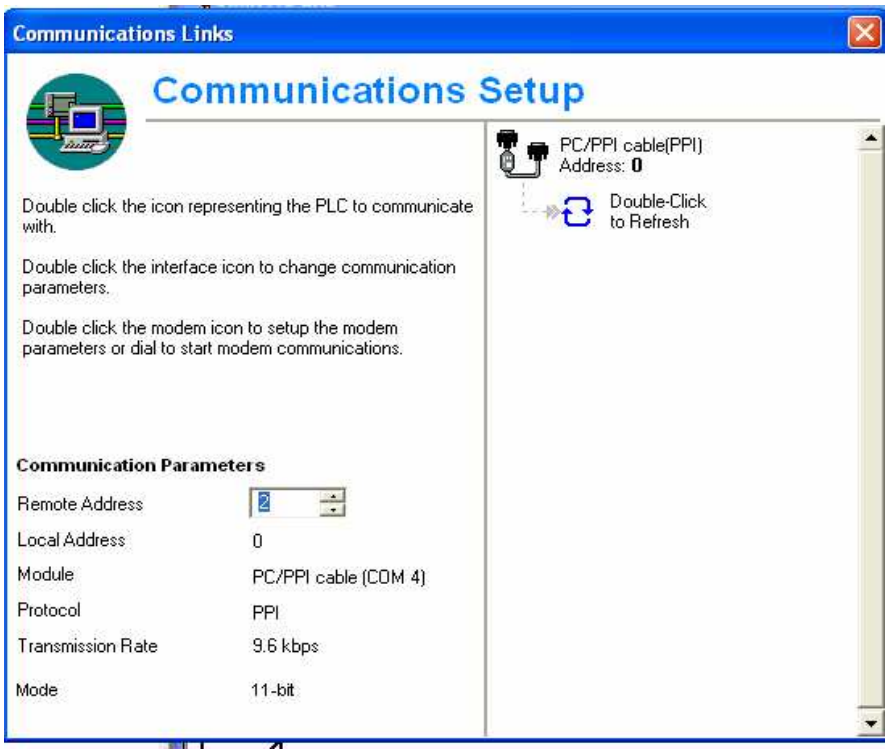



6. Assign the Remote Address ( in this testing, #2 is assigned).

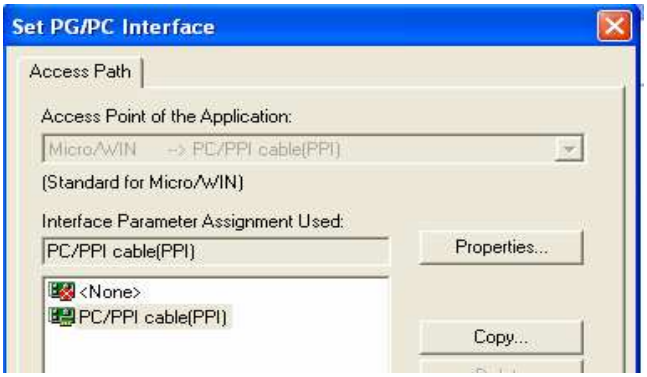



7. Double click

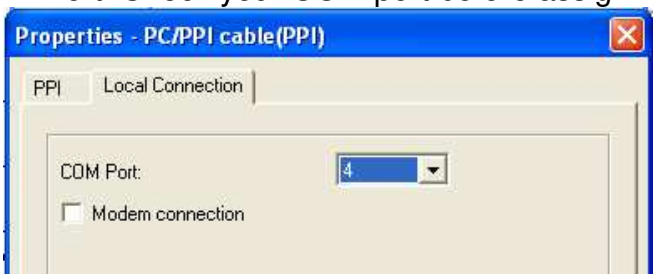



8. In the Access Path, select PC/PPI cable (PPI).

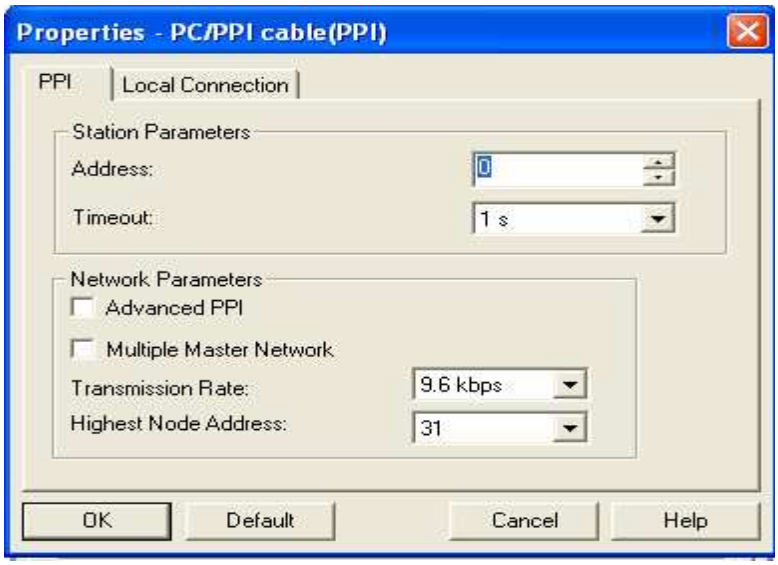
   a) Click the Properties button.




9. Local Connection:
   a) Assign the COM Port. Check your COM port before assigning the number.

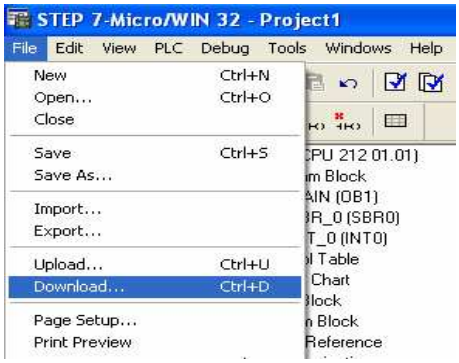



10. PPI :
    a) Assign the station address (#0 is assigned for this test)
    b) Set the transmission rate at 9.6kbps
    c) Highest node address is 31
    d) Click the OK button to exit from the Properties dialogue box

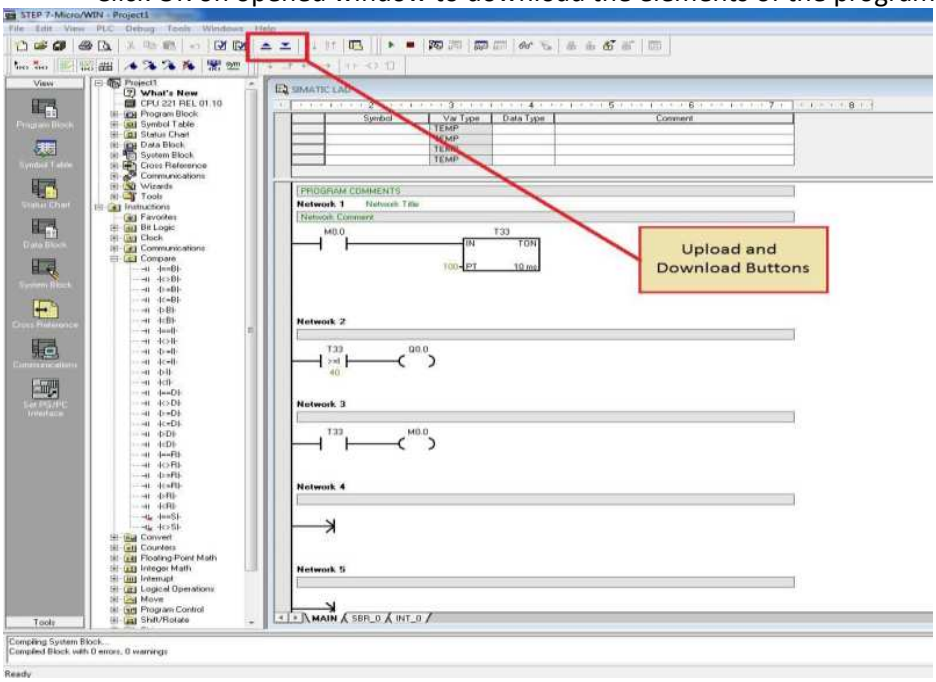



## 5.2 Downloading the Program to PLC

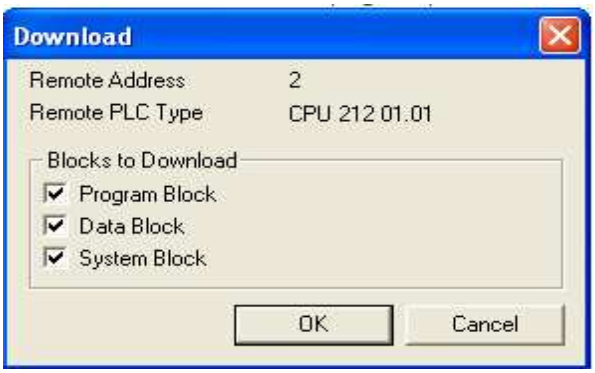
- Click the Download icon on the toolbar or select the **File > Download** menu command to download the program




- Click OK on opened window to download the elements of the program to the S7-200.



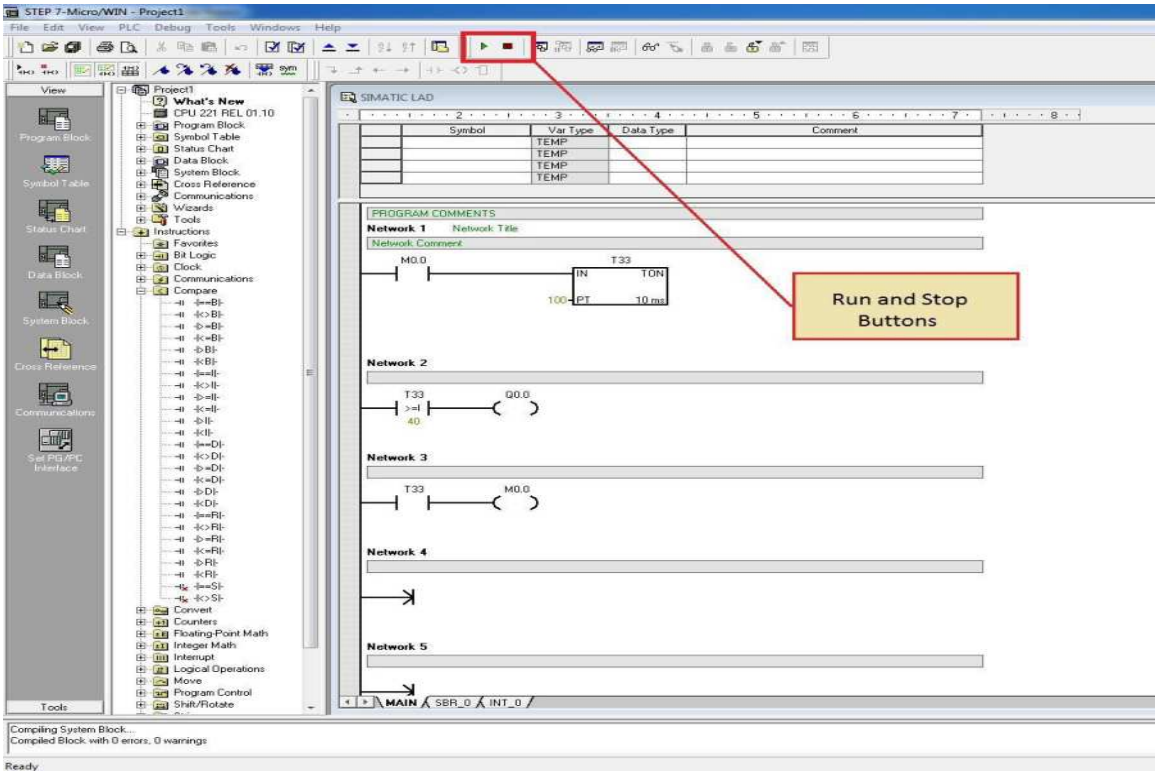
In the Download dialog box, follow all settings as shown in the image below and then click the OK button to download.




### 5.3 Running the PLC

For STEP 7--Micro/WIN to place the S7-200 CPU in RUN mode, the mode switch of the S7-200 must be set to TERM or RUN. When you place the S7-200 in RUN mode, the S7-200 executes the program:

- Click the RUN icon on the toolbar or select the **PLC > RUN** menu command
- Click OK to change the operating mode of the S7-200.



## Problem 1

Control a traffic light system using plc with the following sequence:

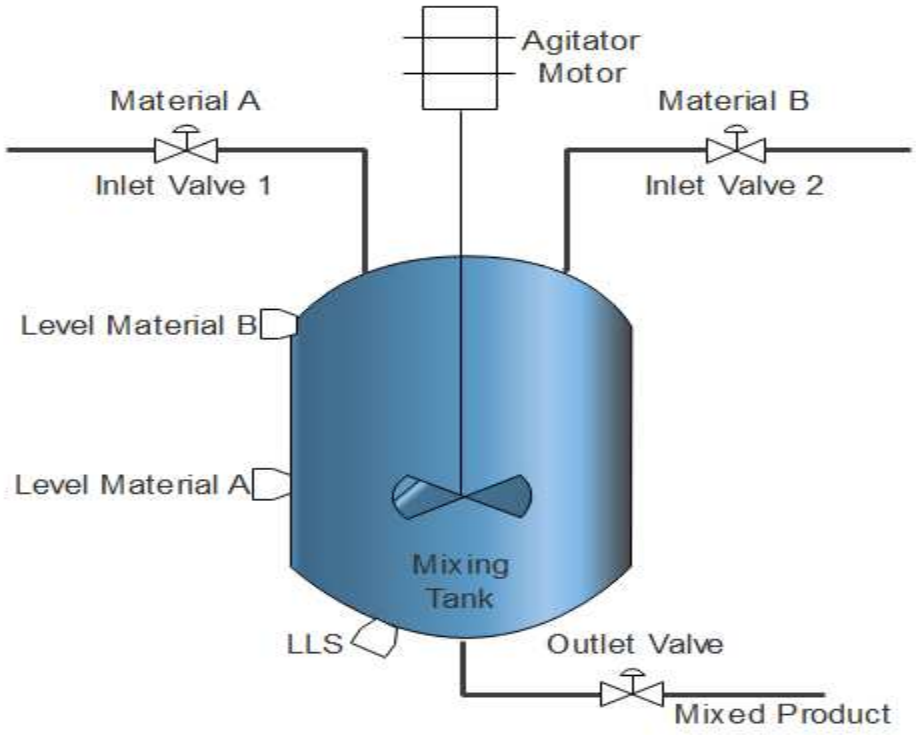
5. The red light is on for a 5 seconds
6. The red light turns off and the yellow light is turned on for a 2 seconds
7. The yellow light turns off and the green light is turned on for a 4 seconds
8. The red light turns on again

## Problem 2

Problem Description

Material A and Material B are collected in a tank. These materials are mixed for a while. Mixed product is then drained out through Outlet valve. Implement this in PLC using Ladder Logic programming language.

Problem Diagram



**Experiment 6** **COUNTERS**

## 6.1 Objectives

After studying this module, the student should be able to:

- List the types of counters used in ladder logic programs;
- Understand how the counters work in ladder logic programs;
- Use status bits of counters to control other instructions;
- Write a ladder logic program using counters.

## 6.2 Theory

A counter allows a number of occurrences of input signals to be counted. A counter is set to same preset number value and, when this value of input pulses has been received, it will operate its contacts. Thus normally open contacts would be closed, normally closed contacts opened.

The S7-200 SIMATIC LAD instruction set includes three types of counters : Count UP Counter(CTU), Count Down Counter (CTD), and Count UP/Down Counter (CTUD).

### 6.2.1 Counter UP (CTU)

The Count Up instruction (CTU) counts up from the current value each time the count up (CU) input makes the transition from off to on. When the current value Cxx is greater than or equal to the preset value PV, the counter bit Cxx turns on. The counter is reset when the Reset (R) input turns on, or when the Reset instruction is executed. The counter stops counting when it reaches the maximum value (32,767).

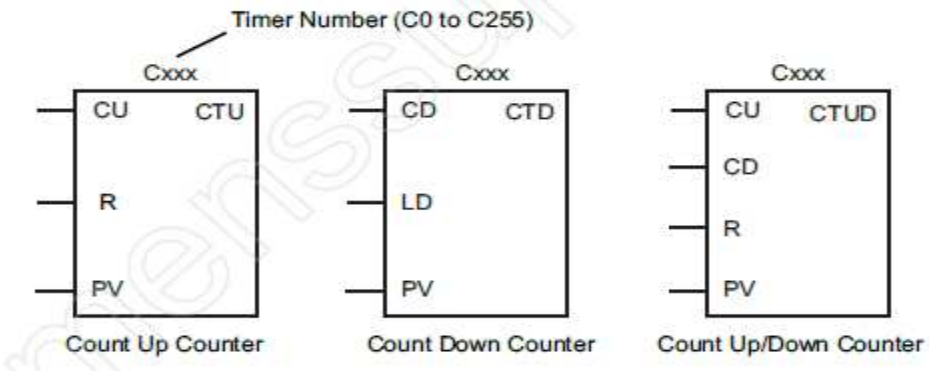
#### 6.2.2 Counter Down (CTD)

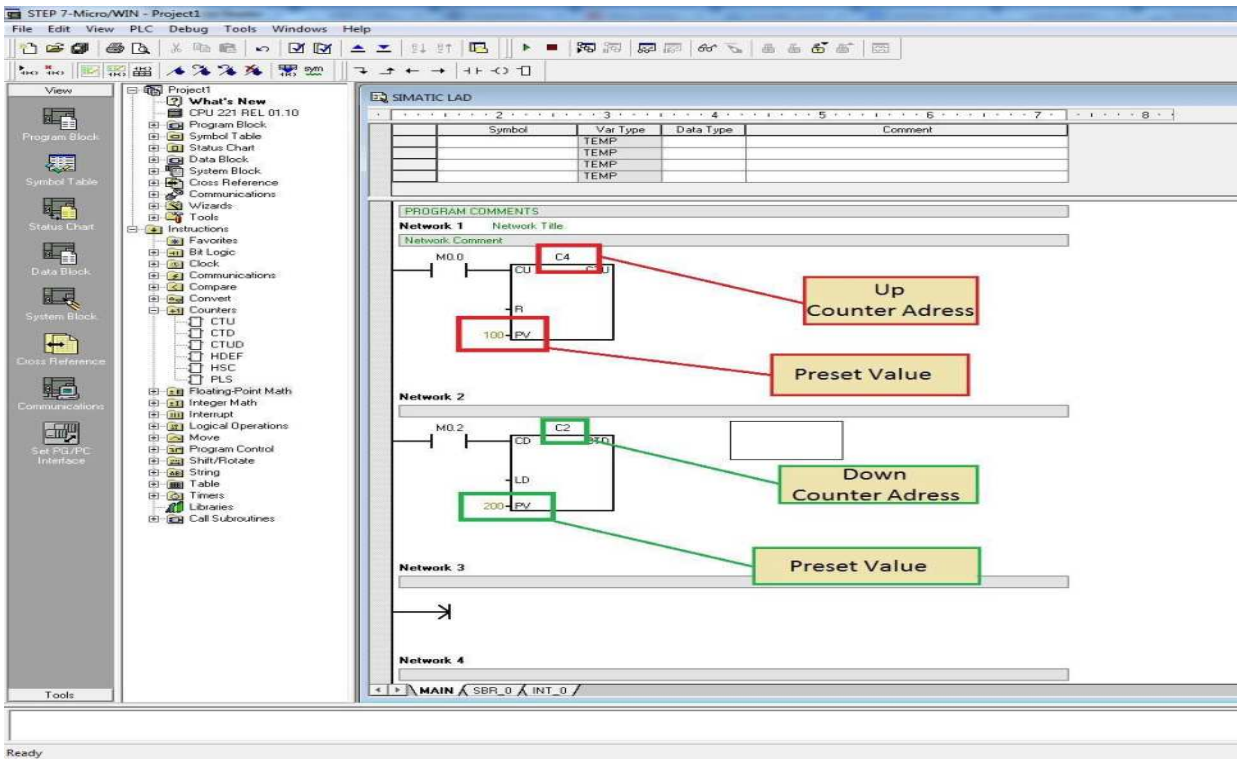
The Count Down instruction (CTD) counts down from the current value of that counter each time the count down (CD) input makes the transition from off to on. When the current value Cxx is equal to 0, the counter bit Cxx turns on. The counter resets the counter bit Cxx and loads the current value with the preset value PV when the load input LD turns on. The counter stops upon reaching zero, and the counter bit Cxx turns on.

R input for CTU and LD input for CTD are reset inputs of counters. You can reset counters by using this inputs.

#### 6.2.3 Counter UO/Down (CTUD)

The counter UP/Down (CTUD) counts up by one each time the count up (CU) input transitions from off to on and count sown by one each time the count down (CD) input transitions from off to on. When the accumulated count equals the preset value (PV), the counter bit turns on. When the reset input (R) turns on or when a Reset instruction is executed, the accumulated count resets to zero and the counter bit turns off.



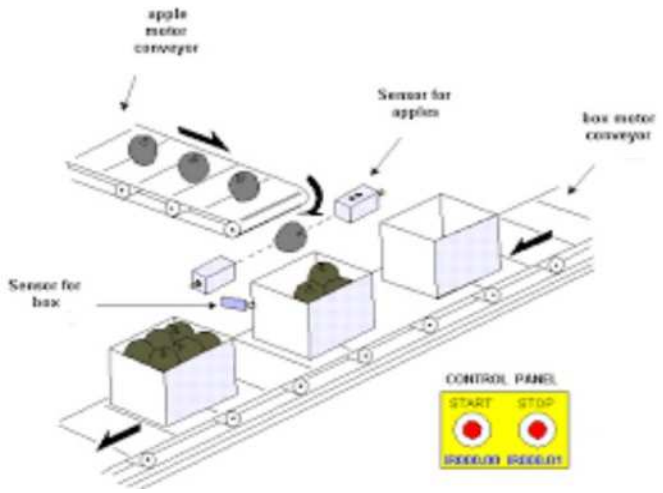




| Type | Operation | Counter Bit | Power Cycle/First Scan |
|---|---|---|---|
| CTU | CU increments the current value.<br>Current value continues to increment until it reaches 32,767. | The counter bit turns on when:<br>Current value >= Preset | Counter bit is off.<br>Current value can be retained.[1] |
| CTUD | CU increments the current value.<br>CD decrements the current value.<br>Current value continues to increment or decrement until the counter is reset. | The counter bit turns on when:<br>Current value >= Preset | Counter bit is off.<br>Current value can be retained.[1] |
| CTD | CD decrements the current value until the current value reaches 0. | The counter bit turns on when:<br>Current value = 0 | Counter bit is off.<br>Current value can be retained.[1] |

Operations of the Counter Instructions

## Problem 1

In this system by pushing start key you activate the power for the system (resetting depends only on a stop key). When started, motor of conveyer for boxes is activated. The conveyer takes a box up to the limit switch, when a box is detected, the motor stops then a conveyer with apples starts moving. Presence of the box allows counter to count 10 apples through a sensor used for apples. Operations repeat until stop key is pressed.

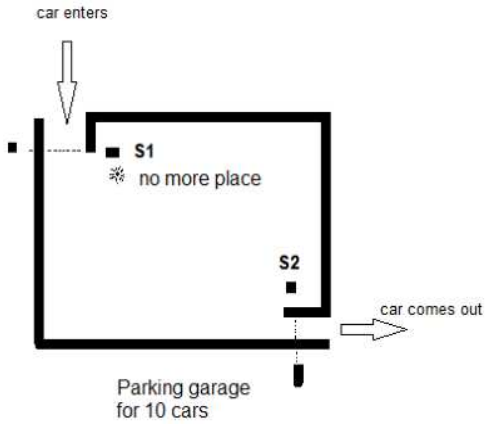


**Experiment 7** **COUNTERS Problems**

## Problem 1

We are dealing with a simple system that can control 10 cars at the maximum. Each time a car enters, PLC automatically adds it to a total sum of other cars found in the garage. Each car that comes out will automatically be taken off. When 10 cars park, a signal will turn on signalizing that a garage is full and notifying other drivers not to enter because there is no space available.

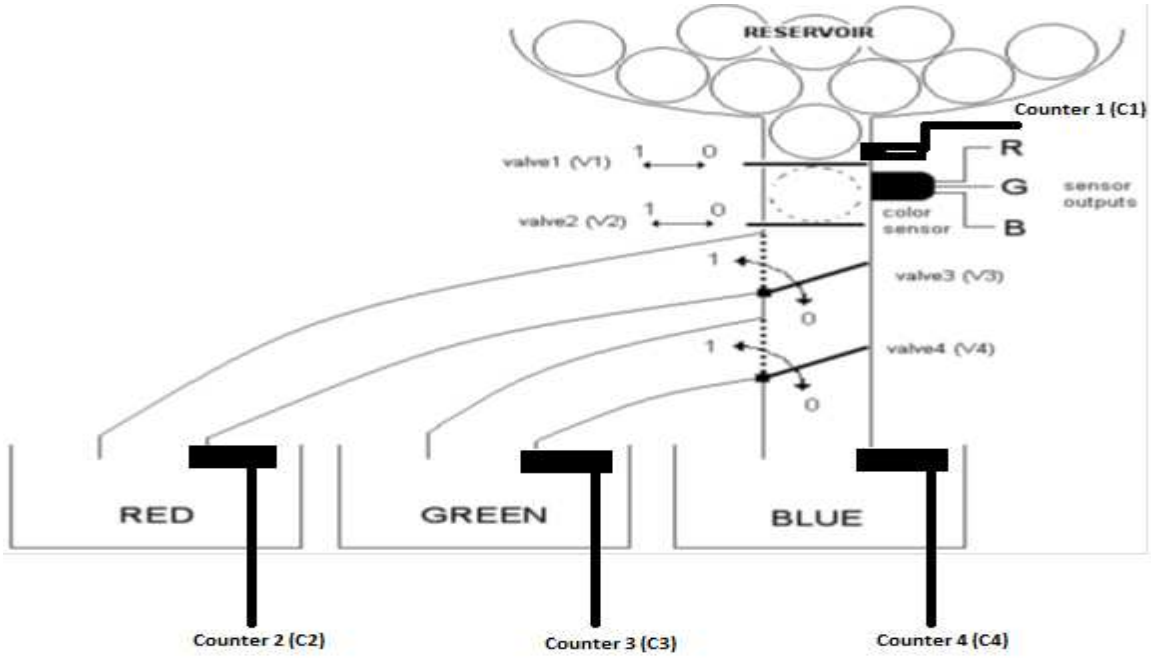
- Construct a LAD for this system.
- Construct input/output table.
- Construct a instruction list for this system



## Problem 2

The Figure below shows a system which separate three different color balls which are mixed in a reservoir and then fill three different labeled with color names bins. The total number of balls is 23 so that you have to count number of felt balls from reservoir by using counter 1 sensor to stop the system when reservoir becomes empty. A color sensor is used to recognize the color of the ball. If there is a red ball in front of the sensor, then " R " output of the sensor becomes 1. "G" output becomes 1, if the ball is green and "B" becomes 1, when the ball is blue. Four valves are used to run the process properly. Directions and input signals of the valves are shown in The Figure. Valve1 (V1) will be opened for 3 seconds then valve 2 (V2) will wait for 5 seconds to sensor measurement can be achieved. After measurement if the ball is red valve 3 (V3) and valve 4 (V4) will be remained on close position. If the ball is green V3 will opened but V4 will remained as closed. Finally if the ball is blue V3 and V4 will be opened. At the end of process we should know the number of balls in each bin (Number of Green Balls, Number of Red Balls and Number of Blue Balls).

- Construct a LAD for this system.
- Construct input/output table.
- Construct a instruction list for this system



**Experiment 8** **PLC Wiring Input and Output**

# PLC Wiring Input / Output

## 8.1 Objectives:

- how to provide ac power to a PLC.
- various types of PLC input configurations.
- how to connect external components to PLC inputs.
- various types of PLC output configurations.
- how to connect PLC outputs to external components

## 8.2 Theory

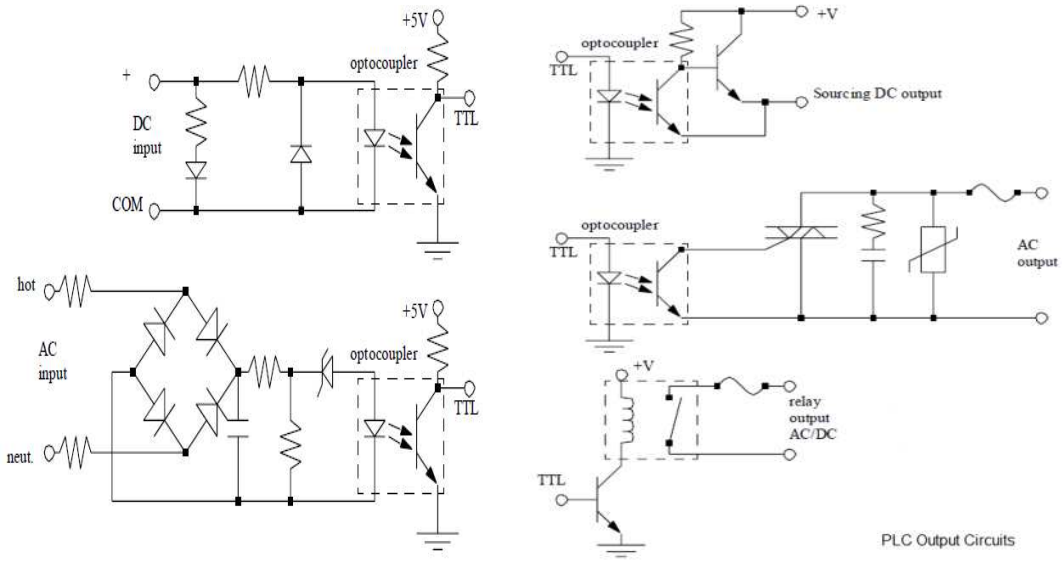
### 8.2.1 INPUTS AND OUTPUTS

Inputs and outputs from a PLC are necessary to monitor and control a process. Both inputs and outputs can be categorized into two basic types: logical or continuous. Consider the example of a light bulb. If it can only be turned on or off, it is logical control.
If the light can be dimmed to different levels, it is continuous.

Outputs from PLCs are often relays, but they can also be solid state electronics such as transistors for DC outputs or Triacs for AC outputs. Continuous outputs require special output cards with digital to analog converters. Inputs come from sensors that translate physical phenomena into electrical signals.

PLC inputs must convert a variety of logic levels to the 5Vdc logic levels used on the data bus. This can be done with circuits similar to those shown below. Basically the circuits condition the input to drive an optocoupler. This electrically isolates the external electrical circuitry from the internal circuitry.

PLC outputs must convert the 5Vdc logic levels on the PLC data bus to external voltage levels. This can be done with circuits similar to those shown below. Basically the circuits use an optocoupler to switch external circuitry. This electrically isolates the external electrical circuitry from the internal circuitry.




PLC Input / Output Circuits

### 8.2.2 INPUTS

In smaller PLCs the inputs are normally built in and are specified when purchasing the PLC. For larger PLCs the inputs are purchased as modules, or cards, with 8 or 16 inputs of the same type on each card.

Input devices such as (Manual Switches, Pushbuttons, Selector Switches, Thermostats, Thermocouple, Pressure Sensors, Liquid level detection, Flow meter, LVDT, Strain gauge, …etc )

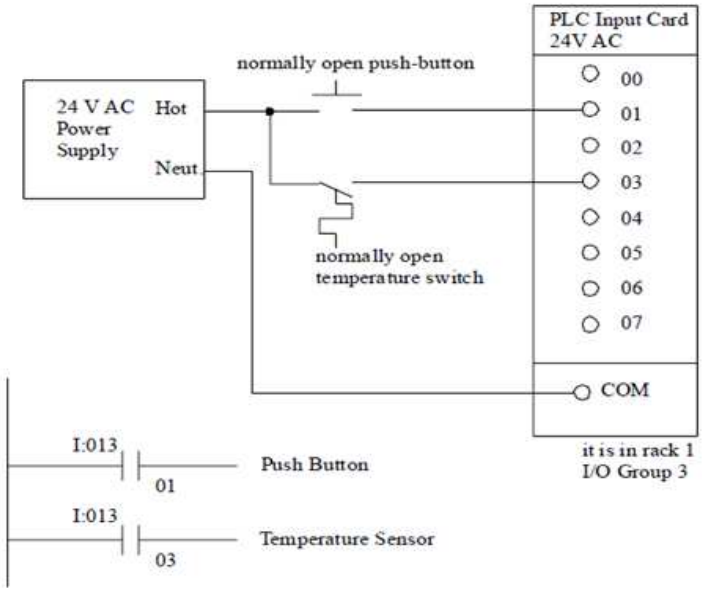
Typical ranges for input voltages

12-24 Vdc / 100-120 Vac / 10-60 Vdc / 12-24 Vac/dc

5 Vdc (TTL) / 200-240 Vac / 48 Vdc / 24 Vac

PLC input cards rarely supply power, this means that an external power supply is needed to supply power for the inputs and sensors.

The example in Figure below shows how to connect an AC input card:




An AC Input and Ladder Logic

In the example there are two inputs, one is a normally open push button, and the second is a temperature switch, or thermal relay. Both of the switches are powered by the hot output of the 24Vac power supply this is like the positive terminal on a DC supply. Power is supplied to the left side of both of the switches. When the switches are open there is no voltage passed to the input card. If either of the switches are closed power will be supplied to the input card.

**Common** (reference voltage) is the neutral on the power supply. In effect we have chosen this to be our 0V reference, and all other voltages are measured relative to it. If we had a second power supply, we would also need to connect the neutral so that both neutrals would be connected to the same common.

### .8.2.3 OUTPUTS

External power supplies are connected to the output card and the card will switch the power on or off for each output.

Output devices such as ( Contactors and Relay, Motors, Pumps, Indication Lamps, Solenoid Vaves, Directional Control Valves)

Typical output voltages are
120 Vac / 24 Vdc / 12-48 Vac /
12-48 Vdc / 5Vdc (TTL) / 230 Vac

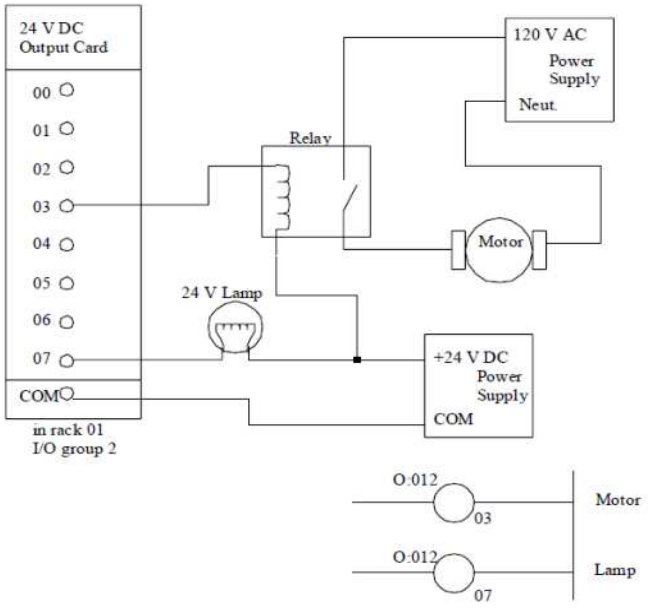
Relays are the most flexible output devices. They are capable of switching both AC and DC outputs. Transistors are limited to DC outputs, and Triacs are limited to AC outputs. Transistor and triac outputs are called switched outputs.

Sourcing and sinking inputs are also popular. This output method dictates that a device does not supply any power.

**Sinking** : When active the output allows current to flow to a common ground. This is best selected when different voltages are supplied.

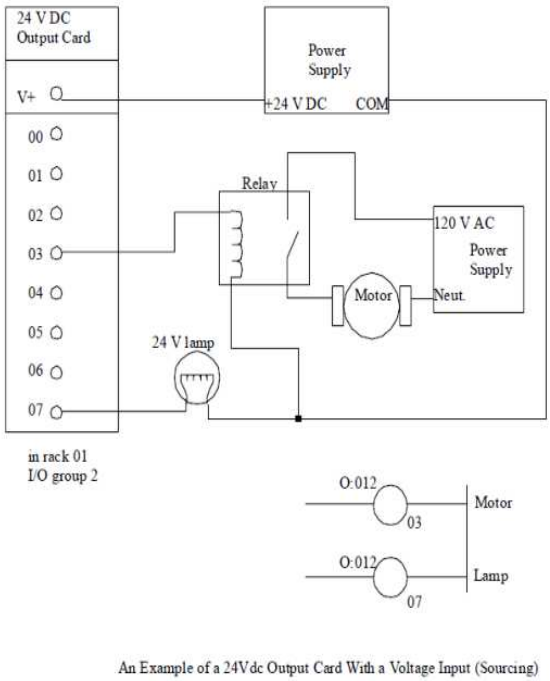
**Sourcing** : When active, current flows from a supply, through the output device and to ground. This method is best used when all devices use a single supply voltage. This is also referred to as NPN (sinking) and PNP (sourcing). PNP is more popular.

The output card shown in figure below is an example of a 24Vdc output card that has a shared common. This type of output card would typically use transistors for the outputs.

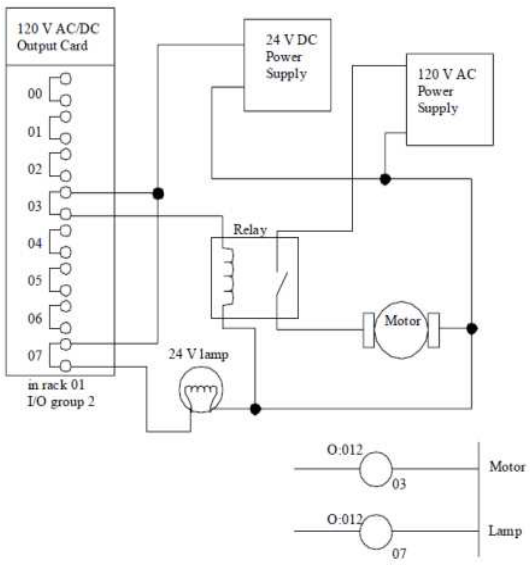



An Example of a 24Vdc Output Card (Sinking)

In this example the outputs are connected to a low current light bulb (lamp) and a relay coil. Consider the circuit through the lamp, starting at the 24Vdc supply. When the output *07* is on, current can flow in *07* to the *COM*, thus completing the circuit, and allowing the light to turn on. If the output is off the current cannot flow, and the light will not turn on. The output *03* for the relay is connected in a similar way. When the output *03* is on, current will flow through the relay coil to close the contacts and supply 120Vac to the motor. This card could have many different voltages applied from different sources, but all the power supplies would need a single shared common.




An Example of a 24Vdc Output Card With a Voltage Input (Sourcing)

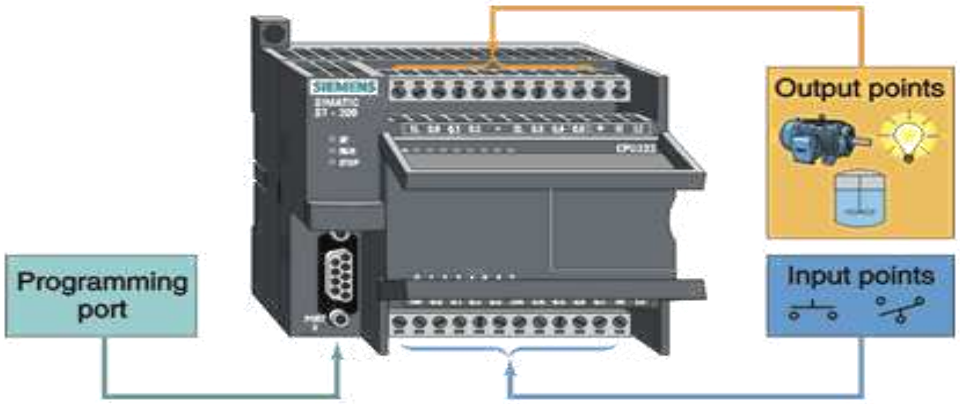



An Example of a Relay Output Card

## 8.3 Analog I/O

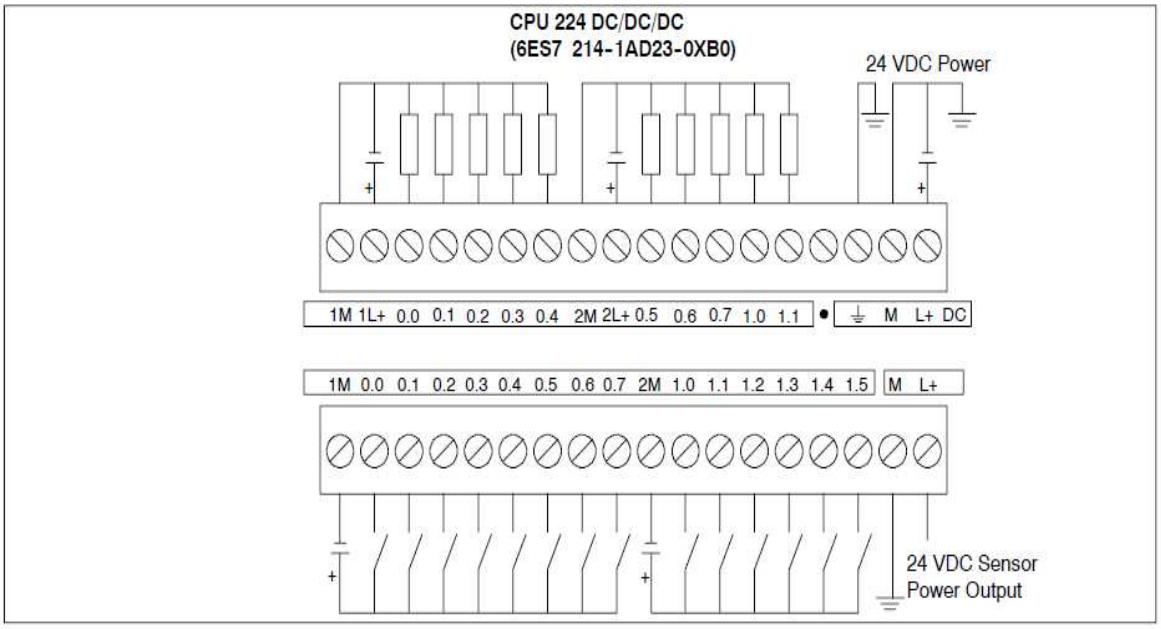
In the early days of programmable logic controllers, processor speed and memory were too limited to support anything but discrete (on/off) control functions. Modern PLC technology, though, is powerful enough to support the measurement, processing, and output of analog (continuously variable) signals. Thus, in order to interface with an analog sensor or control device, some "translation" is necessary between the analog and digital worlds. Inside every analog input module is an ADC, or Analog-to-Digital Converter, circuit designed to convert an analog electrical signal into a multi-bit binary word. Conversely, every analog output module contains a DAC, or Digital-to-Analog Converter, circuit to convert the PLC's digital command words into analog electrical quantities.

Typical analogue signals from ( 0-10 VDC ) or ( 4-20 mA ). They are used to represent changing values such as Voltage, Current, speed, temperature, weight, level, Thermocouple, RTD and Strain gauge

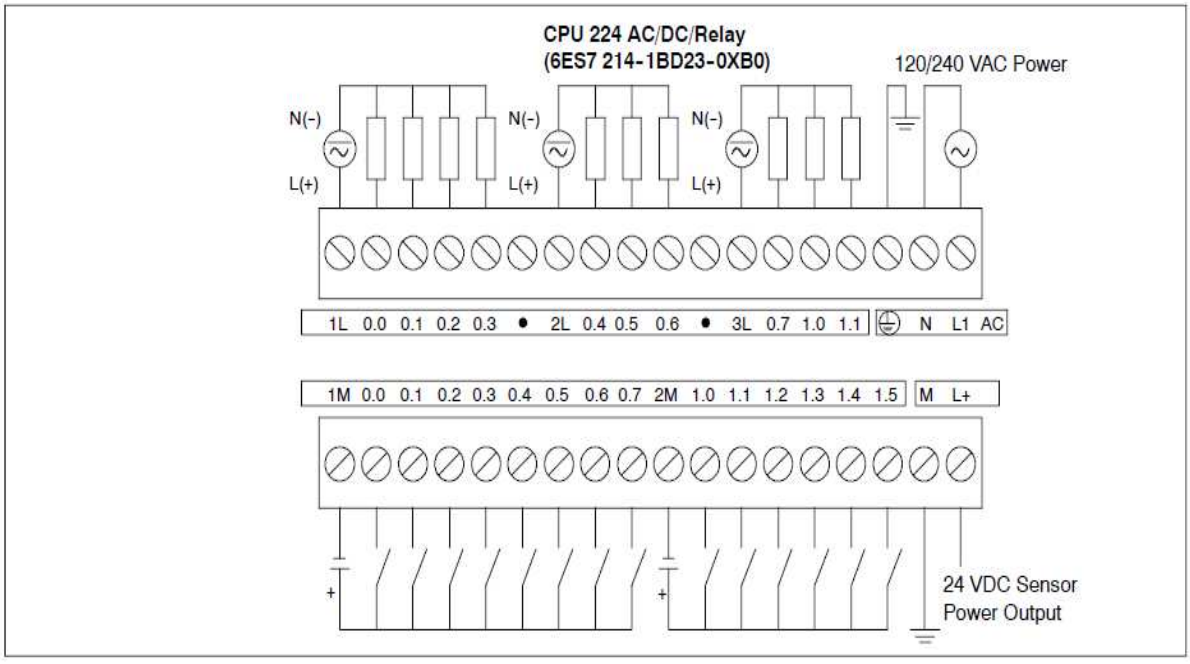


## 8.4 Wiring input/ output S7-200

# CPU 224 DC/DC/DC



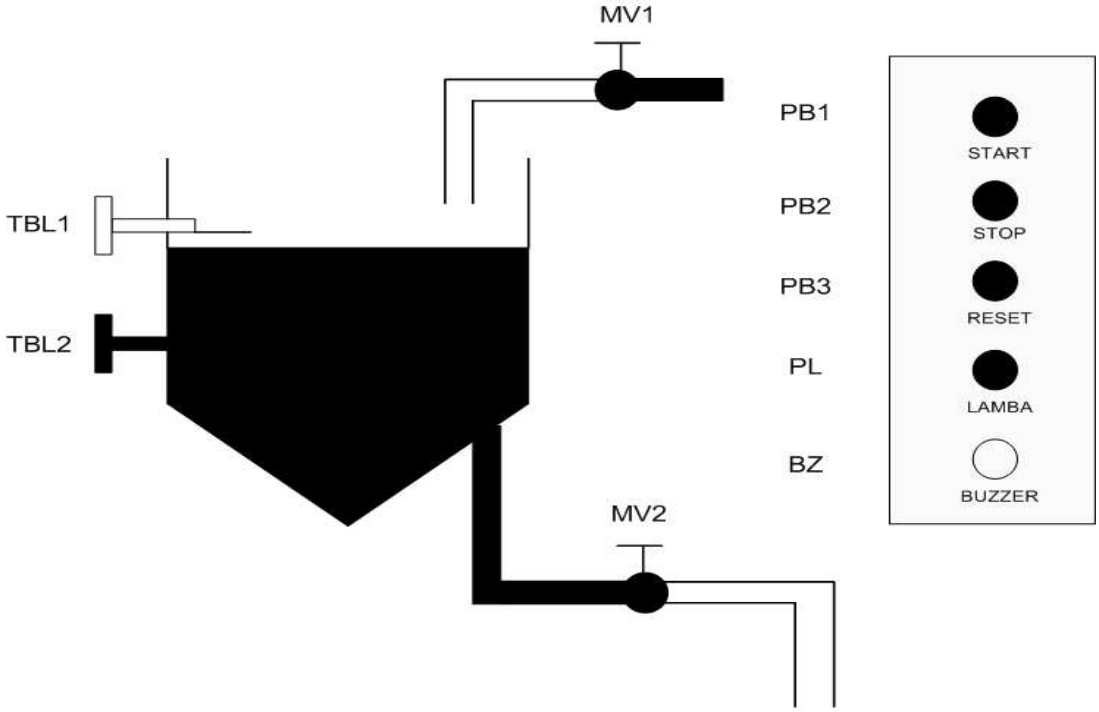

# CPU 224 AC/DC/Relay



## 8.5 Problem

A PLC-controlled system operates as follows. If the Start button is pressed, MV1 will be opened and dye starts to fill the tank. At the same time mixing-motor begins to run. As the level of the dye passes TBL2 and reaches to TBL1, MV1 will be closed and mixing-motor will stop. Then MV2 will be opened and dye begins to run out of the container. After the level of the dye reaches to the below of TBL2, MV2 will be closed. This process is repeated for two (2) times, then the system will stop.
We can observe the stopping of the system with the lamp and the buzzer.
After the system stops, buzzer will continue to run for 2 sec and then it stops. But the lamp continues to light until the reset button is pressed. When the reset button is pressed, the system will be ready to begin the same process again.

- Construct a LAD for this system.
- Construct input/output table.
- Construct a instruction list for this system



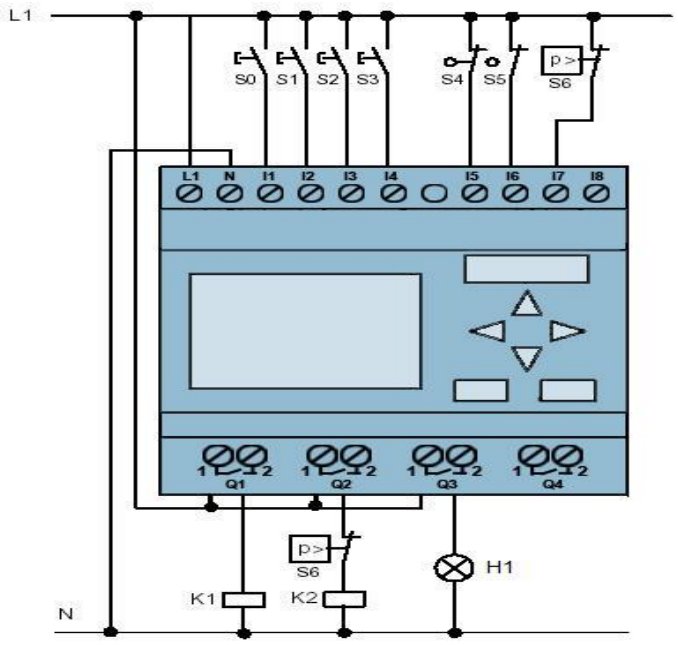

-

**Experiment 9** **LOGO LOGIC**

## 9.1 Theory

The LOGO! logic module has an integrated controller with operator control and display unit. You can use the operator control and display unit of LOGO! to create and edit programs and execute system functions.

You can read in external programs from a program module via the Ethernet interface or a PC cable with the LOGO! Soft Comfort programming software. You can use LOGO! Soft Comfort not only to create a program but also to simulate your circuit on the computer and to output overview diagrams to a printer.




Technology schematic diagram

This PLC shows 8 inputs, 4 outputs, a LCD display screen and a keypad with 6 keys (up, down, right, left and ESC and OK keys). The display screen and keypad presented with the PLC are the viewing elements for states, defaults and other messages. They also are used to programmed the various data values. In standard operational mode, date and time, program version or the number of cycles are displayed.

Keys ▲ ▼ are used to change screen vertically, move within the program, as well as increase or decrease the value of a data.

Keys ◄ ► are used to change screen horizontally, move onto the various characters displayed for a programmed value (tens, units, seconds…).

The **ESC** key is to be used to go back to the previous menu level in case a mistake in entering occurred at one stage.

The **OK** key is used to call for a programmed value, to validate it, or enter the selected menu.

**9.2 Constants and connectors (Co)**

Constants and connectors (= Co) represent inputs, outputs, flags and constant voltage levels

**9.2.1 Inputs:**

**Digital inputs**

Digital inputs are identified with an **I**. The number of the digital inputs (I1, I2, ...) corresponds with the number of the input connectors of the LOGO! Basic and of the connected digital modules, in the order of their installation

**Analog inputs**

We can used **I7** and **I8** inputs to analog signal, which can also be programmed for use as **AI1 … AI8** inputs, analog flags **AM1….AM6**, and analog output **AQ1 and AQ2.**

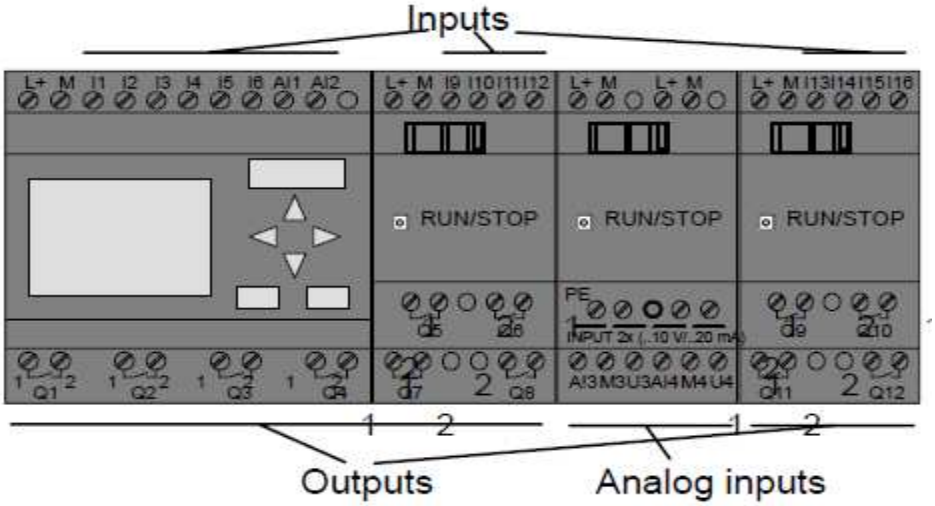
**9.2.2 Outputs:**

**Digital outputs**

Digital outputs are identified by the character **Q**. The output numbers (Q1, Q2, ... Q16) correspond with the numbers of the output connectors at the LOGO!

**Analog outputs**

Analog outputs are identified by the letters **AQ**. Two analog outputs are available, namely AQ1 and AQ2.

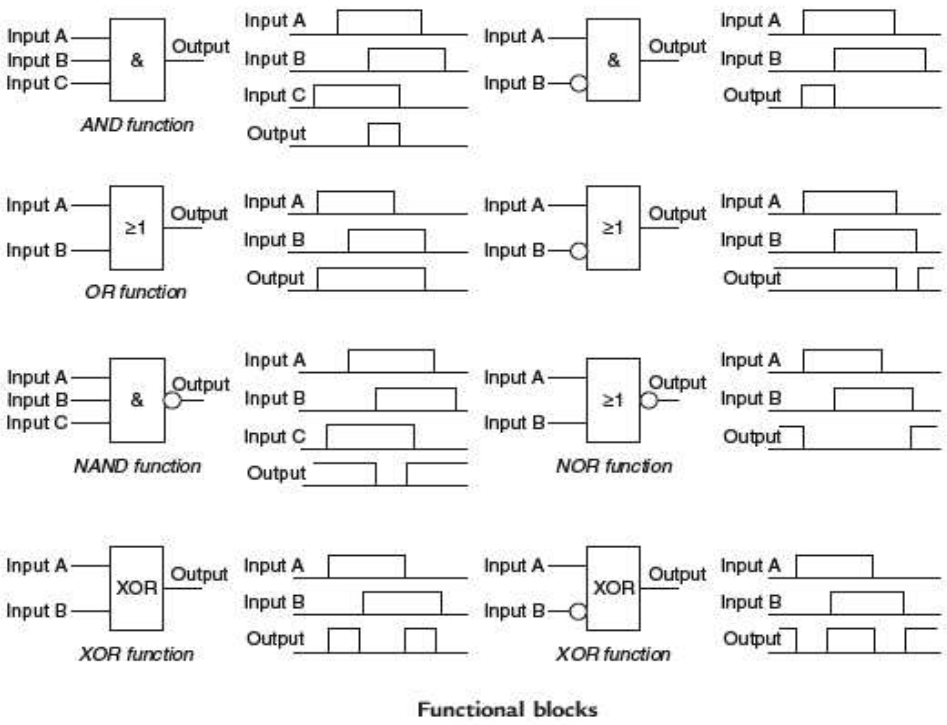



**9.2.3 Flag blocks**

Flag blocks are identified by the letters **M** or **AM** . These are virtual outputs, which output the value of their inputs. LOGO! provides 24 digital flags **M1 ... M24** and 6 analog flags **AM1 ... AM6**.

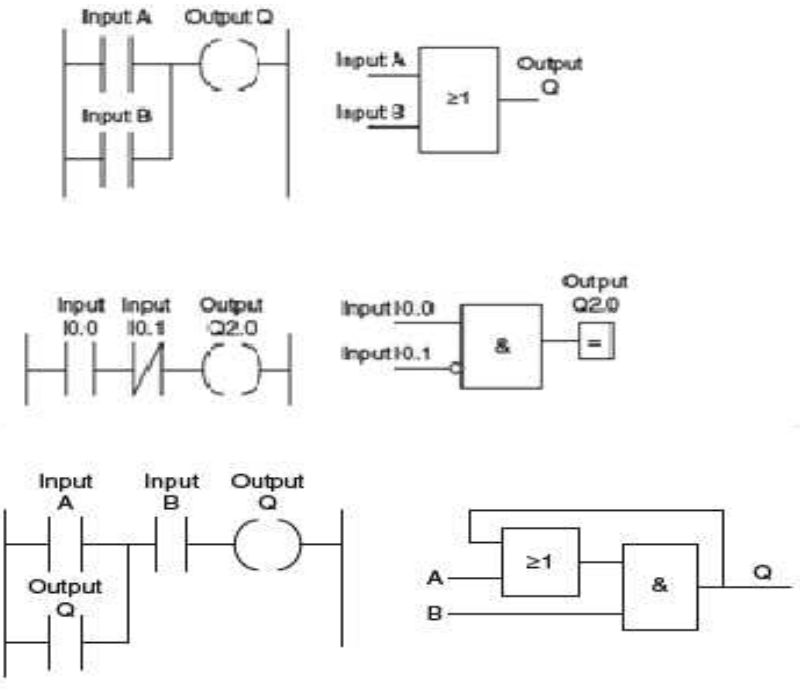
### 9.3 General Function (GF)

Basic functions represent simple logical elements of Boolean algebra. The GF list contains the basic function blocks you can use for your circuit program. The following basic functions are available:




Functional blocks

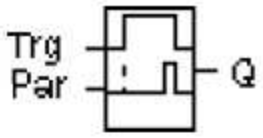
Ladder Diagram and equivalent functional block diagram



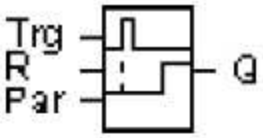

Ladder diagram and equivalent functional block diagram

## 9.4 Special functions list – SF

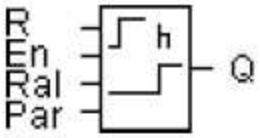
When you create your circuit program in LOGO!, you find the special function blocks in the SF list. The following some of SFs



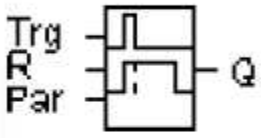

On-delay



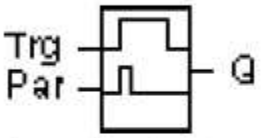

Retentive on-delay



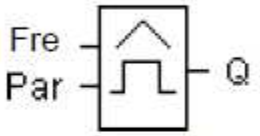

Hours counter



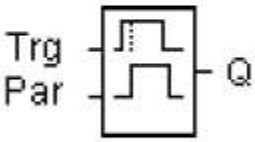

Off-delay



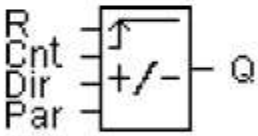

Wiping relay (pulse output)



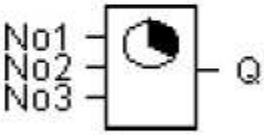

Threshold trigger



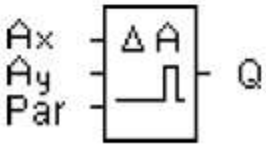

On-/Off-delay



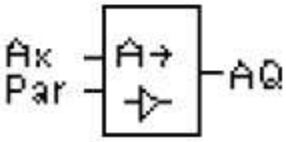

Up/down counter




Weekly timer




Analog comparator




Analog amplifier




Yearly timer

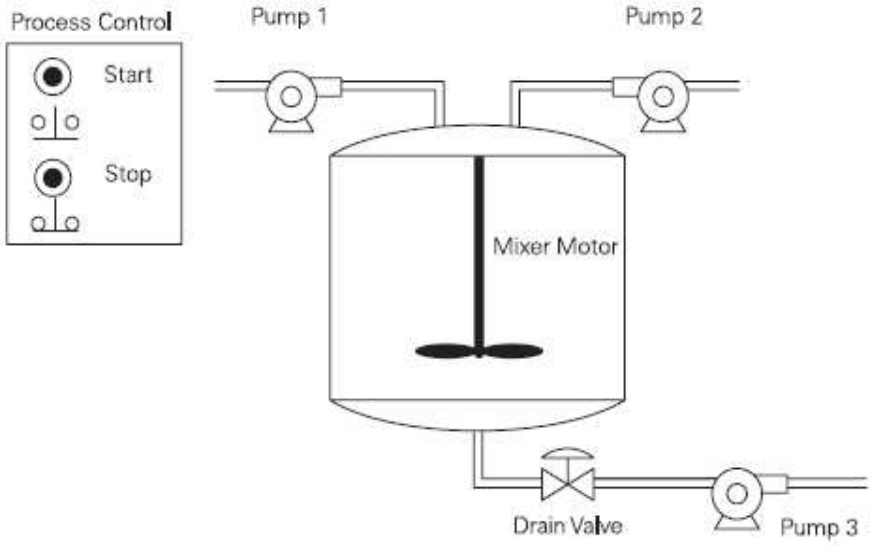
**Experiment 10** **LOGO LOGIC Example**

**Problem 1**

A tank will be filled with two chemicals, mixed, and then drained. When the Start Button is pressed, the program starts pump 1 runs for 5 seconds, filling the tank with the first chemical, then shuts off. The program then starts pump 2, runs for 3 seconds filling the tank with the second chemical. After 3 seconds pump 2 shuts off. The program starts the mixer motor, and mixes the two chemicals for 60 seconds. The program then opens the drain valve, and starts pump 3.

Pump 3 shuts off after 8 seconds and the process stops. A manual Stop switch is also provided to stop process at any time .

- Write a ladder diagram to control the system
- Write program in Functional Block
- Input / output table



**Problem 2**

Control a traffic light system using plc with the following sequence:

1. The red light is on for a 5 seconds
2. The red light turns off and the yellow light is turned on for a 2 seconds
3. The yellow light turns off and the green light is turned on for a 4 seconds
4. The red light turns on again



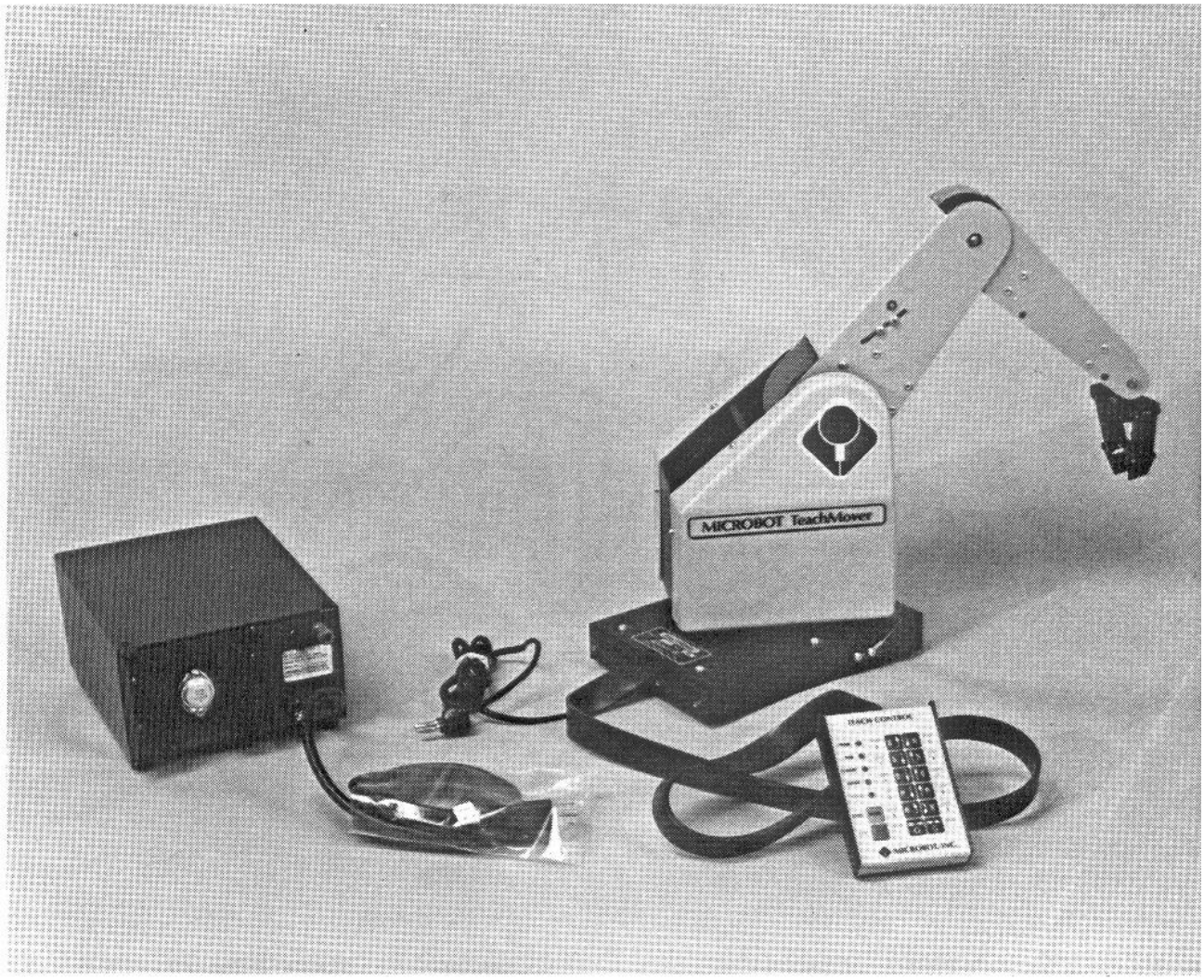

# ALBALQA'A ABLIED UNIVERSITY

# FACULTY OF ENGINEERING TECHNOLOGY

# MECHATRONICS DEPARTMENT

# PLC and ROBOTICS LAB

# MICROBOT TeachMover II



## Experiment One :

## Exercise 1 (Part of Robot)

Theory:

This is Articulated Manipulator, which is a mechanism that can be programmed to move around, The TeachMover system, a robot and a turntable, have been designed to help you get a hands on experience that is as close to an industrial system as is possible.

The electrical motors used in this robot are stepper motor, which is an open loop control system, these motors are controlled by an 8 bit microprocessor which located in the base of the robot, you “talk” to the robot with the teach pendant, the processor listen to the teach pendant and then tell the motor where to go.

The motors are in the body of the robot within the gears. The cables run from the gears to the upper and fore arms to make the move. Figure (1)

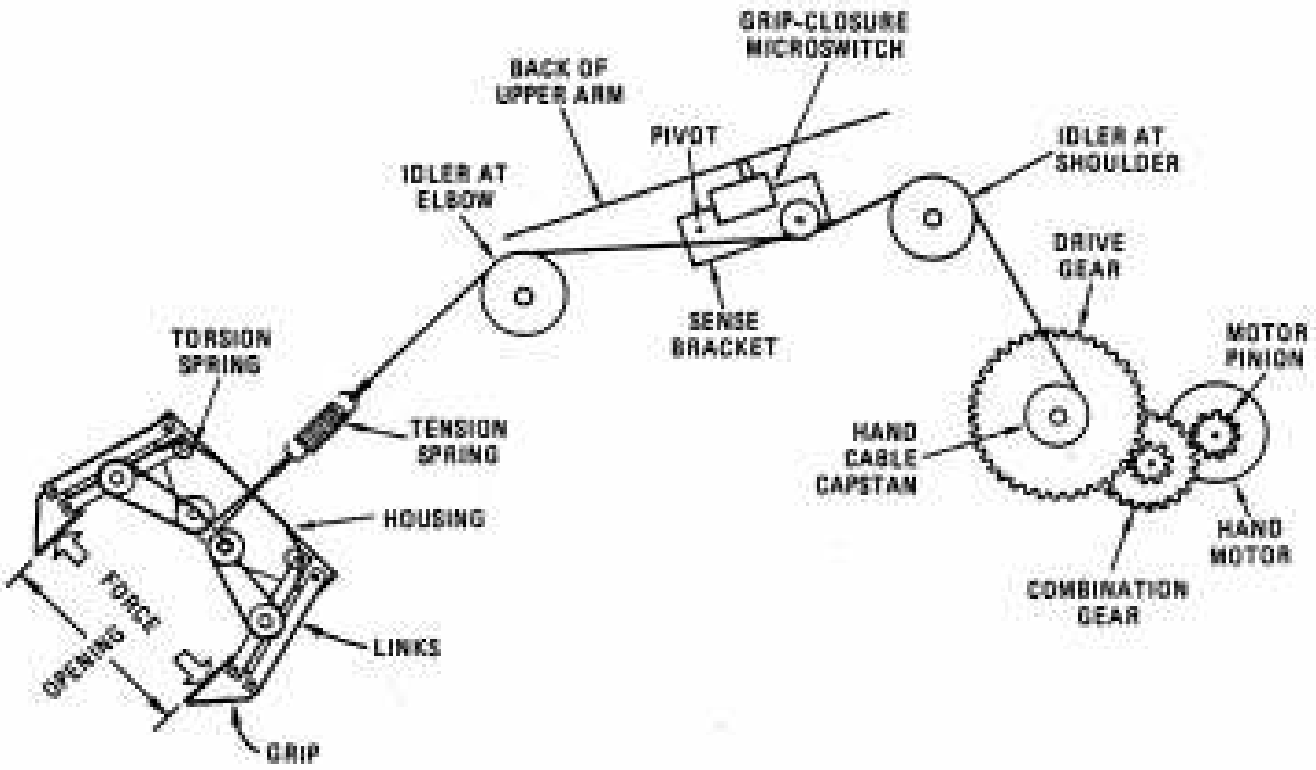



**Figure (1)** Hand cabling diagram

# 1. Teachmover Hardware

## 1.1 Major components

The Teachmover's major structural components are shown in Figure (2). The microprocessor card is housed in the base. The body swivels relative to the base on a hollow shaft attached to the base. This shaft is called the base joint.

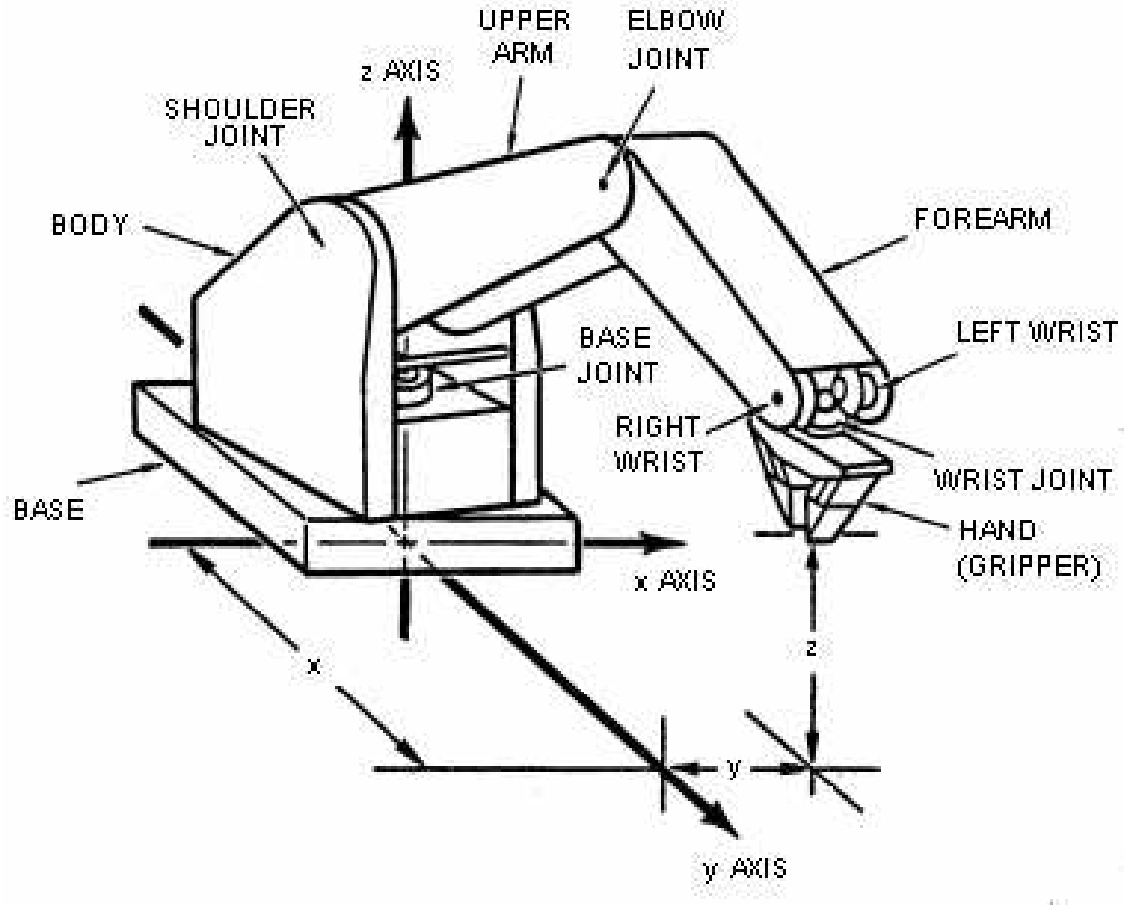



**Figure (2)**

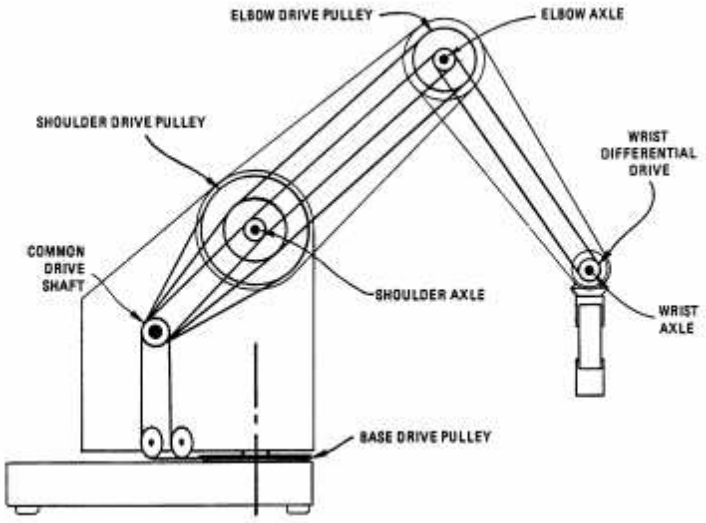
Six stepper motors with gear assemblies are mounted on the body and control each of the six joints. The power wires for the motors pass from the computer card in the base through a hollow shaft to the body. This arrangement provides a direct cable-drive system.

The upper arm is attached to the top of the body and rotates relative to the body on shaft called the shoulder joint. Similarly, the forearm is attached to the upper arm by another shaft know as the elbow joint.

The hand, also called the gripper, is attached to the forearm by two wrist joints. Two separate motors operate wrist joints to control the pitch and roll of the hand.

### 1.2 Cable drive systems

This cable design is an adaptation and refinement of the “tendon technology” used in aircraft, high speed printers and other types of equipment. Each cable is wound around the hub of the drive gear. This serves not only to provide a take-up drum for the cable, but also gives the proper gear-reduction ratios for each of the six drives.




Cable drive system.

- **Base Drive**

The base drive causes the body to rotate on the vertical base axle by driving a large pulley mounted on the base. Two small pulleys, located at the bottom of the body, change the base cable direction so that the cable feed is tangent to the surface of both the drive drum and the base pulley. The termination of the cable is in a clamp fastened by two screws on one side of the base.

- Shoulder Drive

The shoulder cable causes the upper arm to rotate on the horizontal shoulder axle. This cable passes around a drive pulley on the shoulder shaft, and then terminates on the upper arm housing. At the termination point, there are two screws. These screws are used to maintain the cable under tension.

- **Elbow Drive**

The elbow cable causes the elbow to rotate on the horizontal elbow axle. This cable first passes around an idler pulley on the shoulder axle and then around a drive pulley of the same diameter attached to the elbow axle. The cable terminates at a tension mechanism on the forearm housing. If you rotate the elbow manually you will notice that the wrist rotates the same amount in the opposite direction, thus maintaining hand orientation. In rotating the elbow manually you may have noticed something else: when the elbow rotates, the hand opens and closes. Designing cabling to prevent this from happening mechanically would have added undesirable complexity.

- **Wrist Drive**

The right and left wrist cables cause the hand to “roll” and “pitch” relative to the forearm. These cables together control the wrist joint (Figure 12). Both cables pass round idler pulleys on the shoulder axle and the elbow axle, then around the hubs of

bevel gears located on the wrist axle. Tension is maintained in both cables by means of turnbuckles.

## Teach Control

An input device that moves and records the robot position as a program is developed or how you talk to the robot. Most users interface with Teachmovers using the hand-held Teach Control, as shown in Figure (3). Users simply press buttons to move the joints others activate “mode” to enable the control functions and then write simple programs.




**Figure (3) Handheld teach control.**

**B Base S Shoulder E Elbow**

**P Pitch ( Vertical movement of the GRIPPER, Up or Down)**

**R Roll ( Rotational movement either clockwise or counterclockwise)**

**G Grip**

**Lights:**

There are five lights on the teach pendant, the purpose of each is as follow :

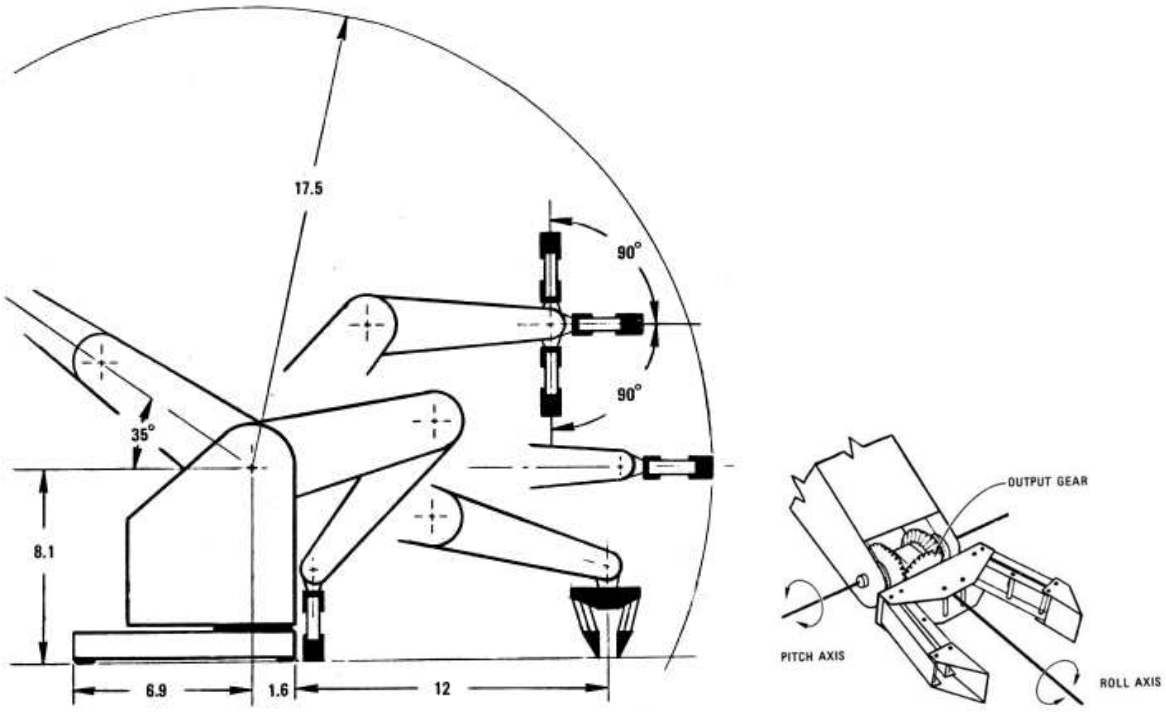
1. TRAIN (GREEN) the light is on when the robot is ready to move, the black keys and lettering are “active” when this green light is on. If the train light is not on, you turn it on by pressing the MODE key first. Then the TRIN key.
2. RUN ( GREEN ) this light is only on when the robot is running a program. This means you have taught it a series of moves and the robot is running the program.
3. CLOSED ( GREEN ) this light is on when the grip is closed. It is turn on when a small switch in the arm of the robot is closed by a cable connected to the gripper fingers.
4. ENTER ( YELLOW) the enter light is yellow because it mean you should have enter a number from the pendant. The number have yellow backgrounds behind black numbers.
5. MODE ( RED ) The MODE light is the “central switching “ place of the pendant. You press the MODE key before you do any other operations. The red light comes on when you press the MODE key.

Note:

- When the **TRAIN** light is on (green), the black keys are working
- The **RED MODE** light activates the red lettered action
- The **YELLOW ENTER** light let you enter constant number

## General Specification of Microbot :

- Configuration : 5 revolution axes and integral hand
- Drive : Electric stepper motor
- Controller : 6502A Microprocessor with 4K bytes of EPROM and 1 K bytes of RAM located in base of unit.
- Interface : Dual RS-232C asynchronous serial communications interfaces ( baud rate is switch-selectable between 110, 150, 300, 600, 1200, 2400, 4800, and 9600 bps)
- Teach control : 14 key 13 function keyboard; 5 output and 7 input bits under computer control




| Motion | Max range of motion | Speed(full load) | Speed ( No load) |
|---|---|---|---|
| Base | ± 90 º | 0.37 rad/sec | 0.42 rad/sec |
| Shoulder | +144 º , -35 º | 0.15 rad/sec | 0.36 rad/sec |
| Elbow | +0 º , -149 º | 0.23 rad/sec | 0.82 rad/sec |
| Wrist Roll | ± 360 º | 1.31 rad/sec | 2.02 rad/sec |
| Wrist Pitch | ± 90 º | 1.31 rad/sec | 2.02 rad/sec |
| Hand | 0-3 in | 8 lb/sec* (35n/sec) | (20mm/sec) |

Table 1. TeachMover performance characteristics

# Exercise 2 : Programming

1. Turn the robot on ( The switch located at the back of robot)
2. Press the REC key in the lower left corner.(save the  home position )
3. Move the robot with the black keys.( anywhere)
4. Press the REC key again ( save the order)
5. Move the robot some more.
6. Press the REC again.
7. Press the MODE key, then the REC ( RED lettering RUN) key .You  just taught the robot three moves and it should now be repeating those moves over and over.
8. Press the MODE key again to stop the robot.

Note:

- To run a program; record a program then press MODE, then RUN.
- To stop a program; press STOP the robot will stop instantly. Or press REC and the robot will stop at the end of the step it is running.

# Exercise 3 (Keys)

To clear the memory between programs, turn the robot off, wait 5 seconds then turn it back on.

**TRAIN** the train key put the robot in the learn mode. When you press it the green light should come on if it is not already on. If it is on the base of the robot will move when you press the key.

**PAUSE** the pause key allows you to enter a pause in the program.

Press the key in the following sequence

**MODE - PAUSE** (yellow light goes on) **– 5 - MODE – TRAIN**

Now train a two more steps.

Press **MODE**. Then **RUN** ( watch it as it run to make sure it paused between the right steps)

**CLEAR** it when you are done ( press and hold down the MODE key, and press the clear key at the same time)

The grip key closes the griper on an object until the grip is closed and then it applies one pound of pressure this is useful in a program if you want your robot to be able to pick up object of different size.

For example

Press **MODE - GRIP** (wait while the fingers closed ) **– MODE –TRAIN**

Add more two move included griper opening, when the robot starts closing its fingers place a panicle so the robot will grip it. Next try a block, bigger than the pencil.

The **OUT** key turn on outputs such as lights and requires entering two number is the number of the output you wish to control. The second number is the condition of the output you want to set. The form is OUT #A ,#B. where A is the number of the output and B is the condition you want A to be. The #B is 1 for on and 0 for off. The light in the teach pendant are output lights you can control.

| OUTPUT | #A | ON | OFF |
|---|---|---|---|
| MODE LIGHT | 0 | 1 | 0 |
| TRAIN LIGHT | 6 | 1 | 0 |
| RUN LIGHT | 7 | 1 | 0 |
| ENTER LIGHT | 8 | 1 | 0 |

For Example Press the following sequence

**MODE – OUT – 8 – MODE -1 – MODE = ( enter light on )**

* This step needs to be a part of a program in the robot

The **MOVE** key allows you to move the robot around without changing the location of the point. This is used to edit a program.

The **STEP** key allows you to have the robot move though a program one a step at the time.

Turn the robot on and record a program with 5 steps. Run the program and have it stop at the beginning point . Then do the following:

**Press MODE – STEP – STEP** …..( The robot will do one step and stop)

The **POINT** key allows you to have the robot do any step you want. Try the following

**Press MODE – POINT – (yellow light on ) – 3 (robot go to step 3 of your program) – MODE - STEP**

**CLEAR** erase the current program

Press the MODE and CLEAR at the same time, make sure the red MODE light BLINK off. Press MODE and hold it down, then press CLEAR.

**The ZERO** clears the registers that keep track of where the motor are. You use this by pressing MODE and ZERO at the same time.

**SPEED** The speed key allows you to change how fast the robot moves. The sequence is

**MODE – SPEED – (enter a number from1-9 )- MODE- TRAIN**

This set the robot at the speed you enter 1 is very slow and 9 is pretty quick.

To move **the turntable**, make sure is connected and then press MODE and TRAIN, at the same time then release the MODE key first, then the TRAIN key. Now the BASE keys BASE keys move the turntable, press REC to remember a position.

# Robotino Robot
# (Mobile Robot System)



## Introduction:

Robotino is a fully functional, high quality mobile robot system with omnidirectional drive. The three drive units allow for motion in all directions forward, backward and sideways, and the robot can be turned on the spot as well. It is also equipped with a webcam and several types of sensors, analogous to distance measurement, for example binary for collision protection and digital to check the actual speed.

## Robotino Features

1. Robotinos controller consists of an embedded microcontroller with a compact flash card and contains several demo applications
2. The controller can be directly accessed via wireless LAN (WLAN). When correctly programmed, Robotino completes assigned tasks autonomously
3. Robotino can be programmed with
   - RobotinoView software at a PC via wireless LAN. RobotinoView is capable of transmitting signals to the motor controller, as well as displaying, changing and evaluating sensor values.
   - Linux and C++ API

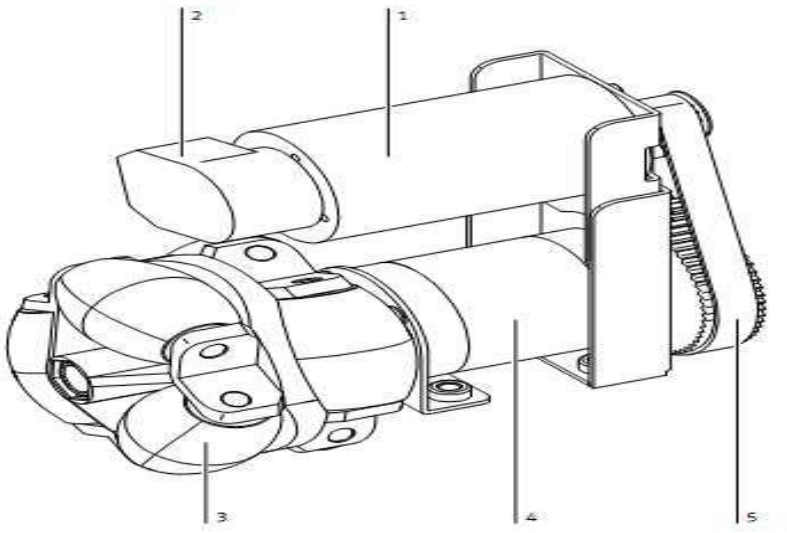
## Robotino Components

**Drive Module:**

Robotino® is driven by three independent, omnidirectional drive units. They are mounted at an angle of 120° to each other.

Each of the 3 drive units consists of the following components:

1. DC motor
2. Gear unit with a gear ratio of 16:1
3. All-way roller
4. Toothed belt
5. Incremental encoder

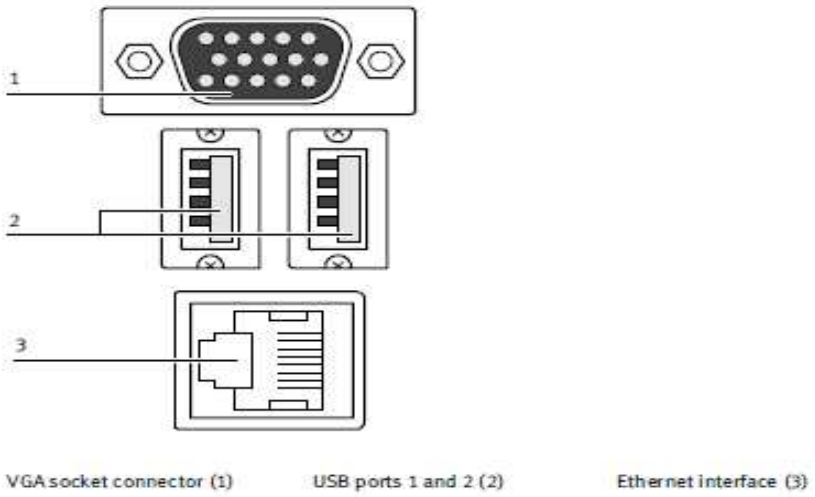



1.Motor 2.Incremental encoder 3.All way roller
4.Gear unit 5.Toothed belt

## Controller Unit

The Robotino® controller unit consists of 3 components:

1. PC 104 processor, compatible with MOPSlcdVE, 300 MHz, and Linux operating system with real-time kernel, SDRAM 128 MB
2. Compact flash card (256 MB) with C++ API for controlling Robotino
3. Wireless LAN access point




Controller Interface

**Sensors**:

1. Infrared Sensors
   - Nine infrared distance measuring sensors which are mounted in the chassis at an angle of 40° to one another
   - The sensors are capable of accurate or relative distance measurements to objects at distances of 4 to 30 cm
2. Incremental Encoder
   - Three encoders are used. Each for an individual motor . They measure the speed in RPM and a PID is used for compensation
3. Anti-collision Sensor (bumper)
   - Switching strip which is secured around the entire circumference of the chassis
4. Inductive Sensors

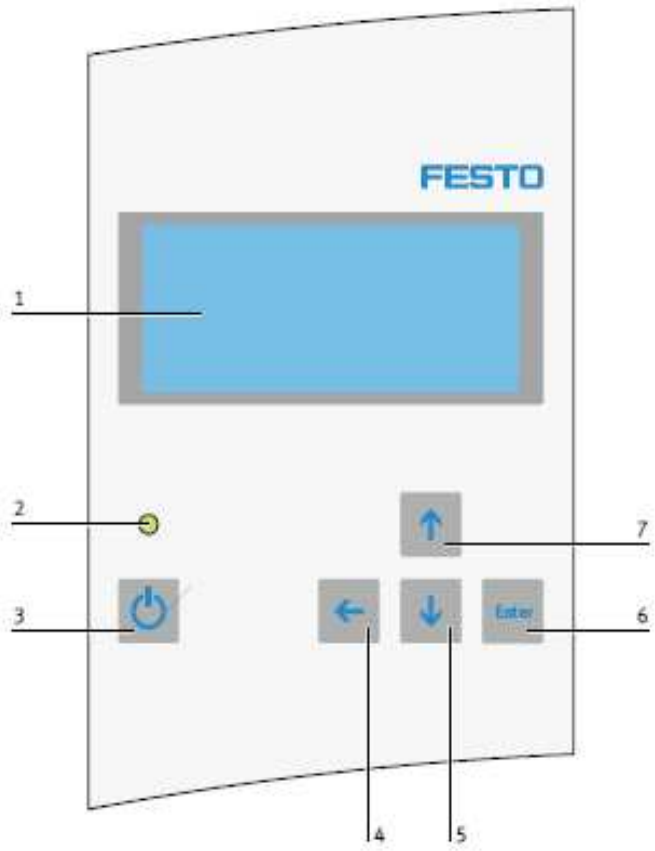



1.Incremental encoder 2.Impact, anti-collision sensors 3.Distance measuring sensors

**User Interface**

It contains the following menu items:

- Languages
- State of charge
- DEMOs
- Network



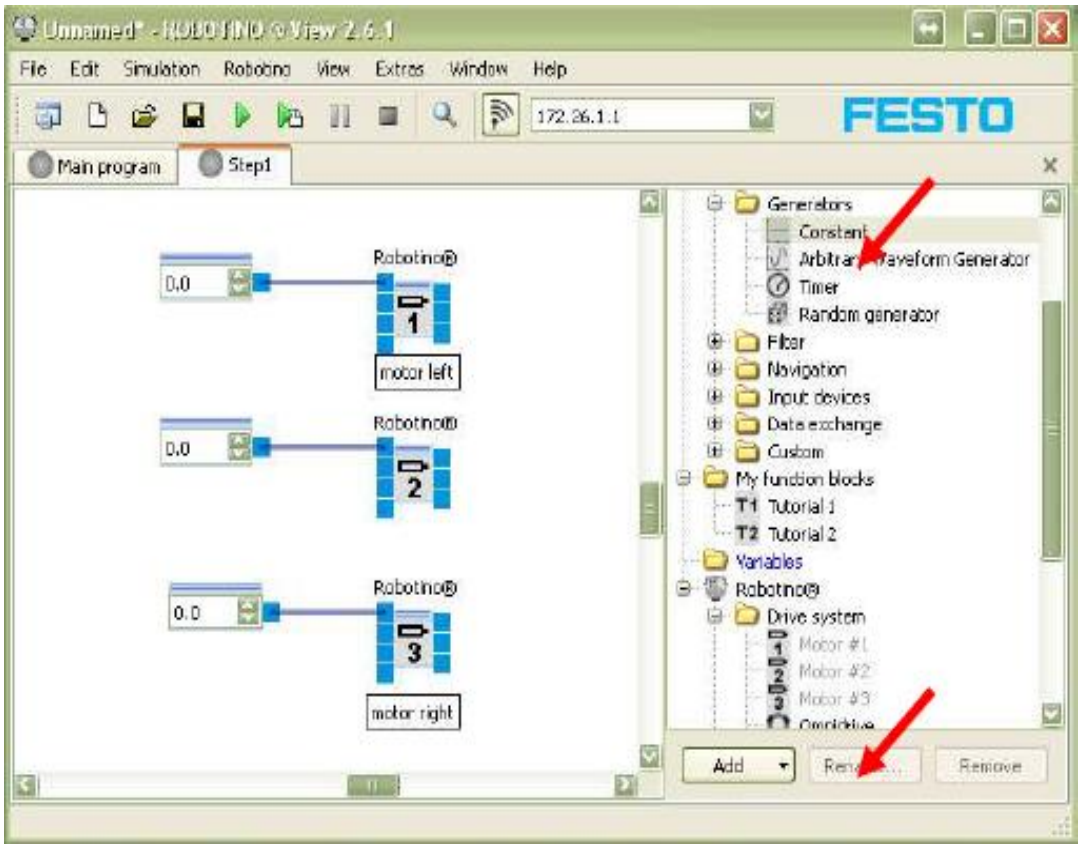

Display (1) LED (2) on/off (3) shift up one menu level (4)
scroll down one selection (5) acknowledge selection (6) scroll up one selection (7)

## Experiment 2 : Robotino view

**Linear travelling of a mobile robot system in any direction**

Actuation of motor, forward movement of robot system

1- Open Robotino view from program menu.
2- Open a blank function block diagram in Robotino View as you see the screen is separated to two areas the first one on the left hand is the programming area the right hand one contains blocks for the robot drive unit, sensors motors and for different logic and mathematic operations.
3- Drag the constant block from generators as shown in the picture.
4- Drag the three motors and then connect the blocks as shown in the following Figure.

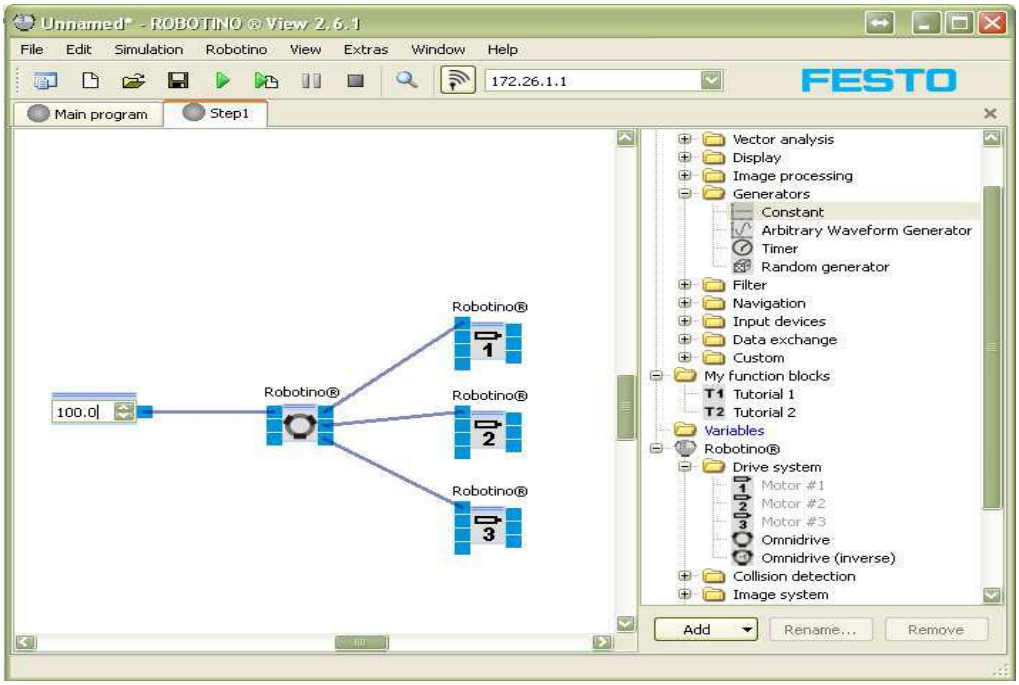



Now move the robot in the forward direction what do you notice about the three motors speed and direction?

What is the constants suitable values that moves the robot in the forward direction. Note that the speed is in rpm and the direction is indicated by negative sign. Let the speed be 100 rpm now set the suitable values for the constants block and then start connection by pressing on the button shown in the following figure.
sure that the IP address for the Robotino is correct.
When it converts to green instead of gray that means the connection is establish now run the program by pressing on start main program button avoid collision and press stop when needed

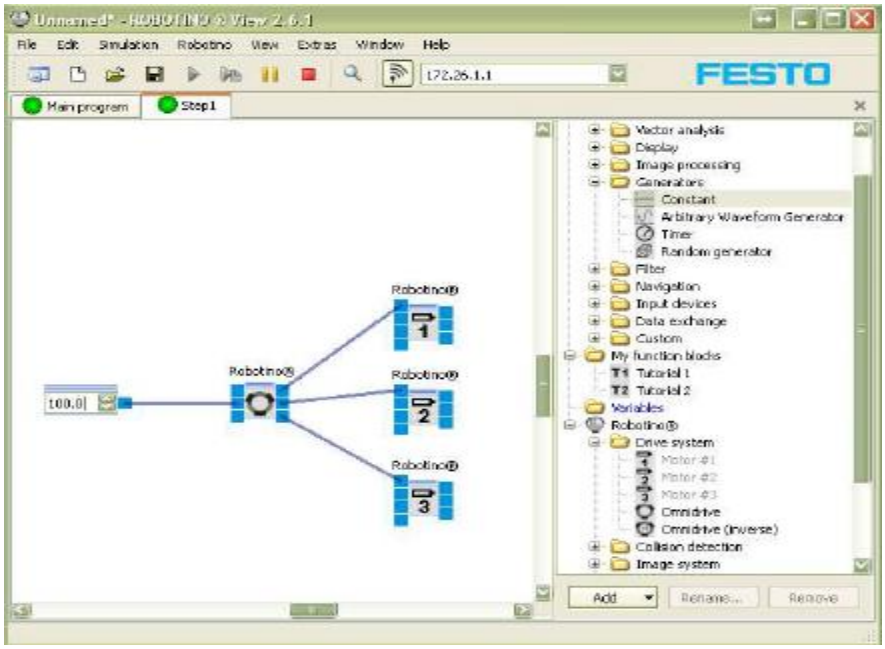
**Omnidrive**

1- Open a new blank function block diagram in robotino view.
2- drag the three motors and omnidrive block(from robotino-drive system) and constant from generators and connect the blocks as in the following figure

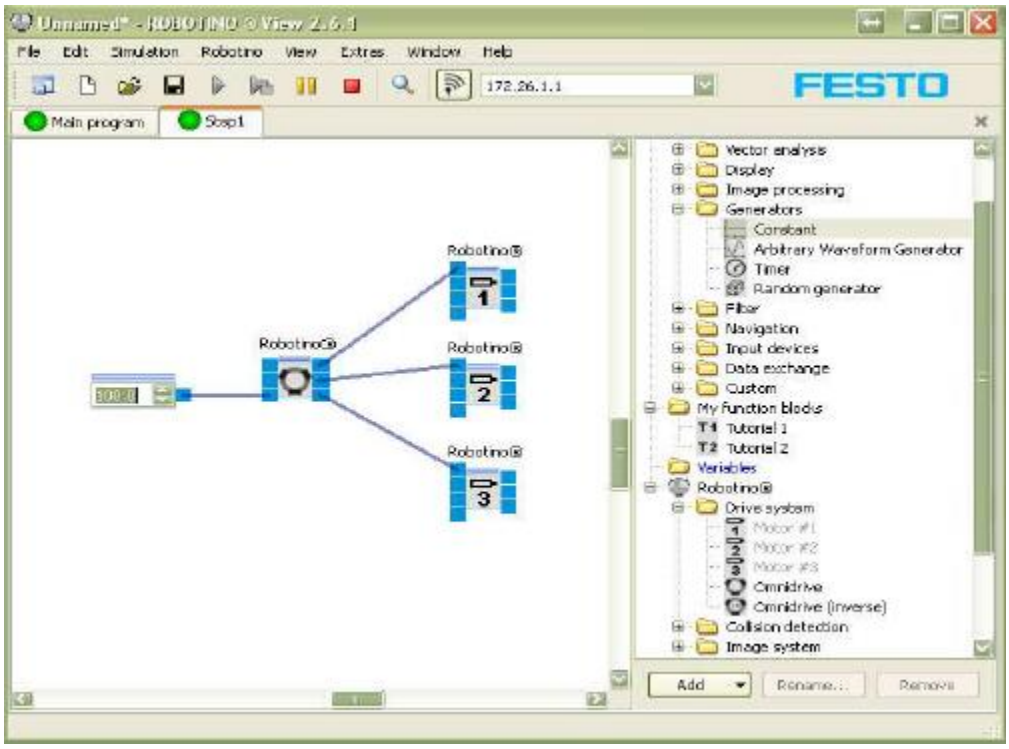



Right 100 in the constant block this means 100 mm/s. run the program and notice the motion.
Change the constant value to -100 mm/s and notice the motion again.

Now connect between the blocks as in the following figure:



Run the program, notice the motion then change the constant value to -100 and run the program again.
Again connect the blocks as follows:

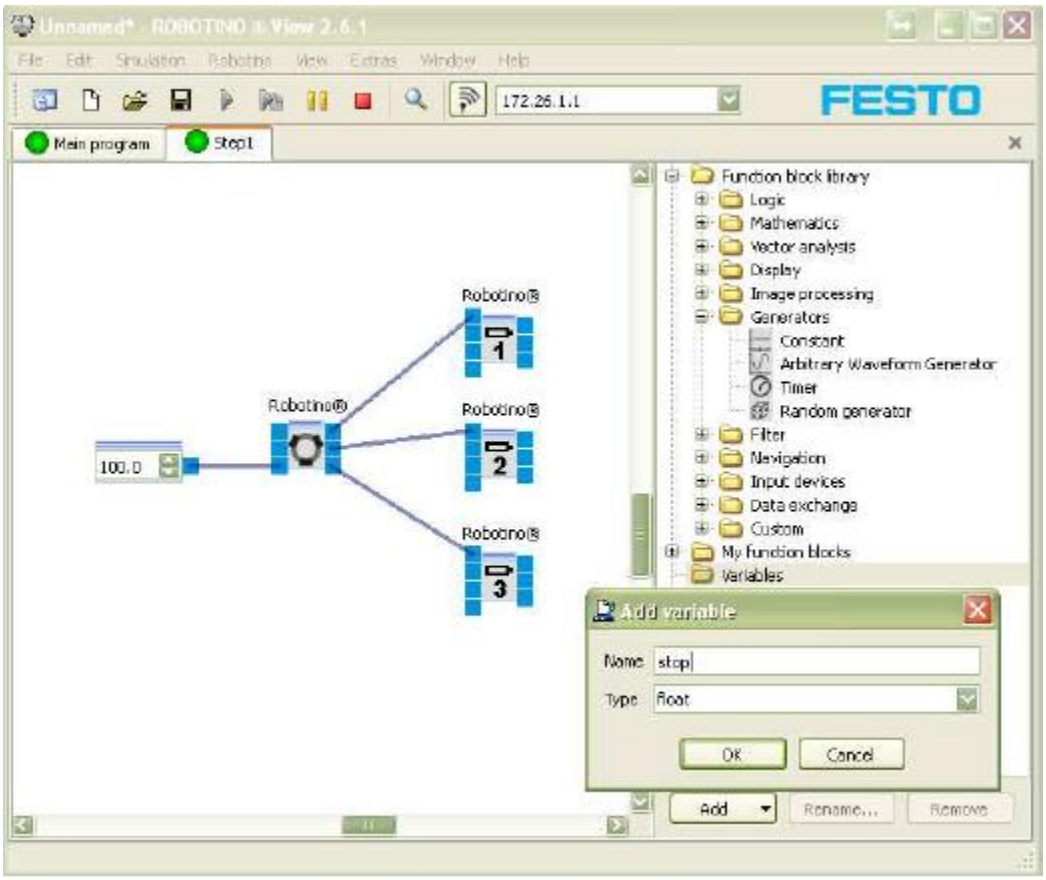



Run the program and notice the motion, change the value to -100 and notice the motion again.

**Exercise: Now let the robot moves in 45o with speed 100mm/s.**

Now you are going to move the robotino in the forward direction and add collision protection to the program.

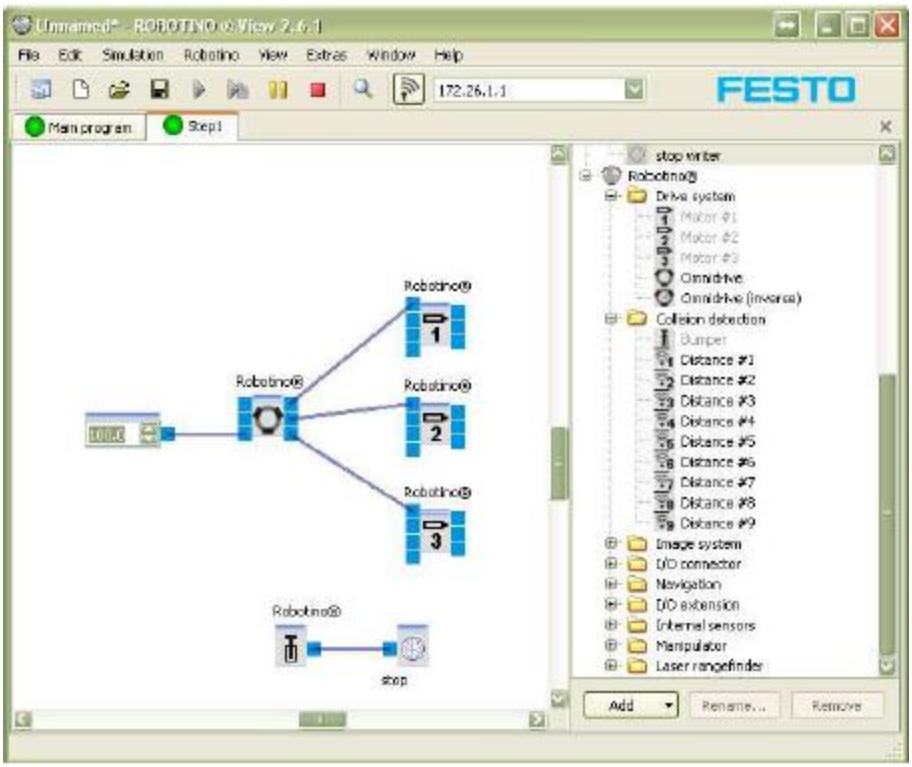
1- open the forward program, on the right menu click right on the variables the following window will appear .



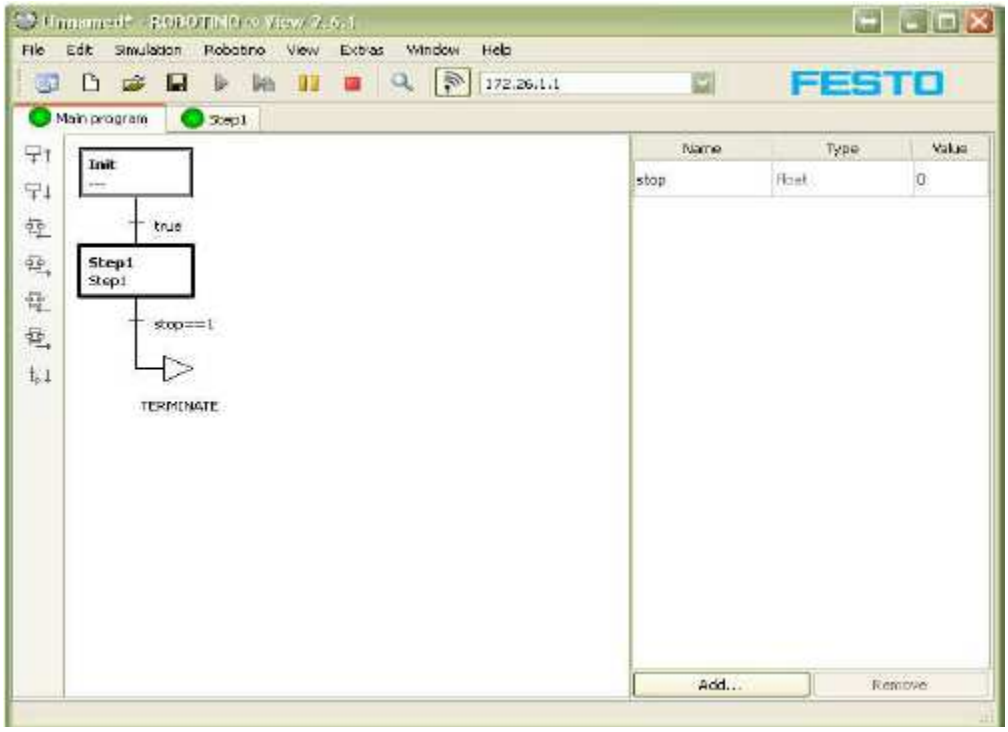

Write stop for the name then click ok.

Two blocks will appear stop reader and stop writer, drag the stop writer block to the blocks function window.

Drag bumper block from robotino collision detection menu and connect between the blocks as in the following picture.




Now open the main program on the left of the function block window change false to stop==1and initial to TERMINATE.




Run the program and notice what happens.